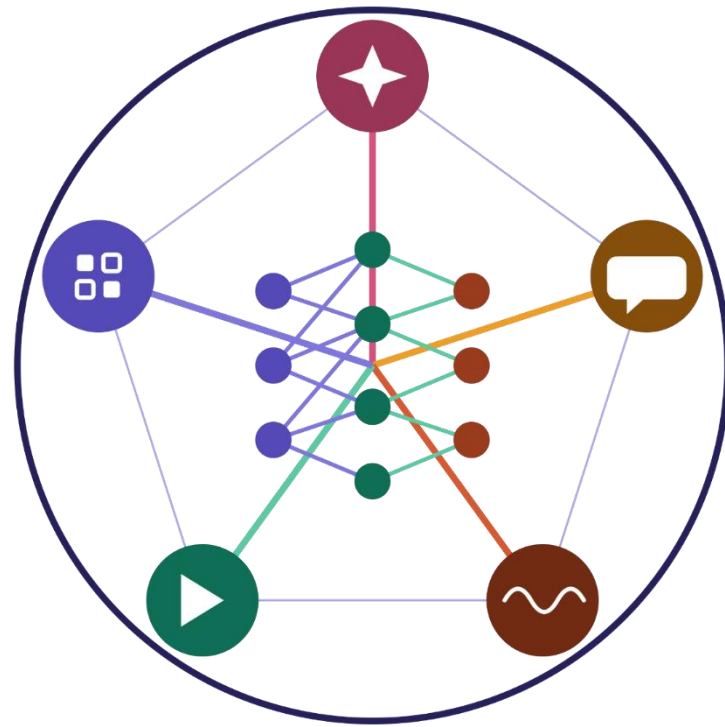




Lecture 05

Fitting Neurons with Gradient Descent and Automatic Differentiation

Let's improve upon the
perceptron & learn about
a neural network model
for which the training
always converges

Our Goals

- A learning rule that is more robust than the perceptron: always converges even if the data is not (linearly) separable
- Combine multiple neurons and layers of neurons ("deep neural nets") to learn more complex decision boundaries (because most real-world problems are not "linear" problems!)
- Handle multiple categories (not just binary) in classification
- Do even fancier things like generating NEW images and text

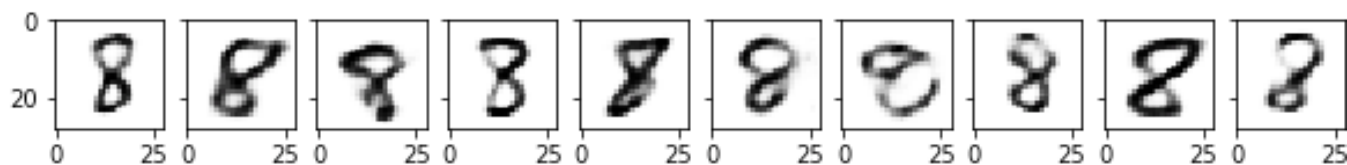
← This lecture

← Next lecture(s)

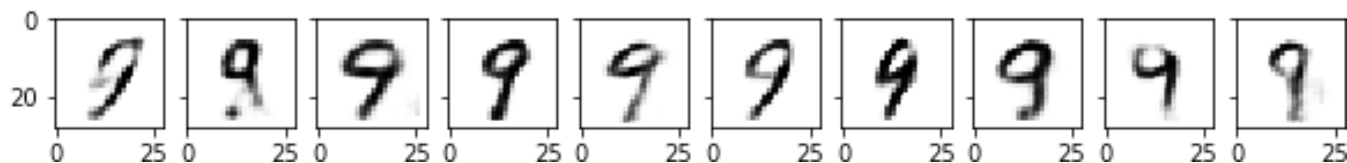
← Next lecture(s)

← More towards the end of the course

Class Label 8



Class Label 9



All based on the same learning algorithm and extensions thereof. So, this is prob. the most fundamental lecture!

Good news:

- After this lecture, there won't be any "new" mathematical concepts.
- Everything in DL will be extensions & applications of these basic concepts.

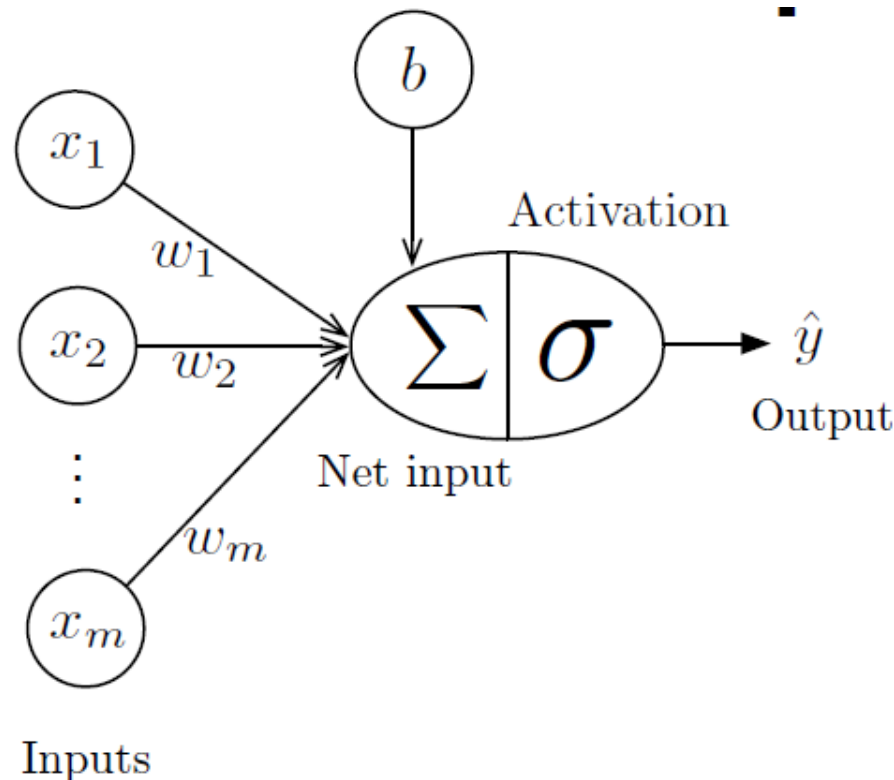
Lecture Overview

1. Online, batch, and minibatch mode
2. Relation between perceptron and linear regression
3. An iterative training algorithm for linear regression
4. Understanding gradient descent
5. Training an adaptive linear neuron (Adaline)
6. PyTorch Resources
7. Computation Graphs
8. Automatic Differentiation in PyTorch
9. Training ADALINE Manually Vs Automatically in PyTorch
10. A Closer Look at the PyTorch API

Training neural nets: Online, batch, and minibatch modes

1. **Online, batch, and minibatch mode**
2. Relation between perceptron and linear regression
3. An iterative training algorithm for linear regression
4. Understanding gradient descent
5. Training an adaptive linear neuron (Adaline)
6. PyTorch Resources
7. Computation Graphs
8. Automatic Differentiation in PyTorch
9. Training ADALINE Manually Vs Automatically in PyTorch
10. A Closer Look at the PyTorch API

Perceptron Recap



$$\sigma \left(\sum_{i=1}^m x_i w_i + b \right) = \sigma (\mathbf{x}^T \mathbf{w} + b) = \hat{y}$$

$$\sigma(z) = \begin{cases} 0, & z \leq 0 \\ 1, & z > 0 \end{cases}$$

$$b = -\theta$$

Let $\mathcal{D} = (\langle \mathbf{x}^{[1]}, y^{[1]} \rangle, \langle \mathbf{x}^{[2]}, y^{[2]} \rangle, \dots, \langle \mathbf{x}^{[n]}, y^{[n]} \rangle) \in (\mathbb{R}^m \times \{0, 1\})^n$

1. Initialize $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m, \mathbf{b} := 0$

2. For every training epoch:

A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$:

(a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$ ← Compute output (prediction)

(b) $err := (y^{[i]} - \hat{y}^{[i]})$ ← Calculate error

(c) $\mathbf{w} := \mathbf{w} + err \times \mathbf{x}^{[i]}, b := b + err$ ← Update parameters

General Learning Principle

Let $\mathcal{D} = (\langle \mathbf{x}^{[1]}, y^{[1]} \rangle, \langle \mathbf{x}^{[2]}, y^{[2]} \rangle, \dots, \langle \mathbf{x}^{[n]}, y^{[n]} \rangle) \in (\mathbb{R}^m \times \{0, 1\})^n$

"On-line" mode

1. Initialize $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m$, $b := 0$
2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$:
 - (a) Compute output (prediction)
 - (b) Calculate error
 - (c) Update $\mathbf{w}, b$

This applies to all common neuron models and (deep) neural network architectures!

There are some variants of it, namely the "batch mode" and the "minibatch mode" which we will briefly go over in the next slides and then discuss more later

Let $\mathcal{D} = (\langle \mathbf{x}^{[1]}, y^{[1]} \rangle, \langle \mathbf{x}^{[2]}, y^{[2]} \rangle, \dots, \langle \mathbf{x}^{[n]}, y^{[n]} \rangle) \in (\mathbb{R}^m \times \{0, 1\})^n$

"On-line" mode

1. Initialize $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m, \mathbf{b} := 0$
2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$:
 - (a) Compute output (prediction)
 - (b) Calculate error
 - (c) Update $\mathbf{w}, b$

Batch mode

1. Initialize
2. For every training epoch:
 - A. Initialize $\Delta \mathbf{w} := 0, \Delta b := 0$
 - B. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$:
 - (a) Compute output (prediction)
 - (b) Calculate error
 - (c) Update $\Delta \mathbf{w}, \Delta b$
 - C. Update $\mathbf{w}, b$:
$$\mathbf{w} := \mathbf{w} + \Delta \mathbf{w}, b := b + \Delta b$$

Let $\mathcal{D} = (\langle \mathbf{x}^{[1]}, y^{[1]} \rangle, \langle \mathbf{x}^{[2]}, y^{[2]} \rangle, \dots, \langle \mathbf{x}^{[n]}, y^{[n]} \rangle) \in (\mathbb{R}^m \times \{0, 1\})^n$

"On-line" mode

1. Initialize $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m, \mathbf{b} := 0$
2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$:
 - (a) Compute output (prediction)
 - (b) Calculate error
 - (c) Update $\mathbf{w}, b$

In practice, we usually shuffle the dataset prior to each epoch to prevent cycles

Batch mode

1. Initialize
2. For every training epoch:
 - A. Initialize $\Delta \mathbf{w} := 0, \Delta b := 0$
 - B. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$:
 - (a) Compute output (prediction)
 - (b) Calculate error
 - (c) Update $\Delta \mathbf{w}, \Delta b$
 - C. Update $\mathbf{w}, b$:
$$\mathbf{w} := \mathbf{w} + \Delta \mathbf{w}, b := \mathbf{b} + \Delta b$$

Let $\mathcal{D} = (\langle \mathbf{x}^{[1]}, y^{[1]} \rangle, \langle \mathbf{x}^{[2]}, y^{[2]} \rangle, \dots, \langle \mathbf{x}^{[n]}, y^{[n]} \rangle) \in (\mathbb{R}^m \times \{0, 1\})^n$

"On-line" mode

1. Initialize $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m$, $b := 0$
2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$:
 - (a) Compute output (prediction)
 - (b) Calculate error
 - (c) Update $\mathbf{w}, b$

In practice, we usually shuffle the dataset prior to each epoch to prevent cycles

"On-line" mode II (alternative)

1. Initialize
2. For for t iterations:
 - A. Pick random $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$:
 - (a) Compute output (prediction)
 - (b) Calculate error
 - (c) Update $\mathbf{w}, b$

No shuffling required

(actually, not really stochastic because a fixed training set instead of sampling from the population)

Let $\mathcal{D} = (\langle \mathbf{x}^{[1]}, y^{[1]} \rangle, \langle \mathbf{x}^{[2]}, y^{[2]} \rangle, \dots, \langle \mathbf{x}^{[n]}, y^{[n]} \rangle) \in (\mathbb{R}^m \times \{0, 1\})^n$

Minibatch mode

(mix between on-line and batch)

1. Initialize $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m$, $b := 0$
2. For every training epoch:

The most common mode in deep learning. Any ideas why?

- A. Initialize $\Delta \mathbf{w} := \mathbf{0}$, $\Delta b := 0$
- B. For every $\{\langle \mathbf{x}^{[i]}, y^{[i]} \rangle, \dots, \langle \mathbf{x}^{[i+k]}, y^{[i+k]} \rangle\} \subset \mathcal{D}$
 - (a) Compute output (prediction)
 - (b) Calculate error
 - (c) Update $\Delta \mathbf{w}$, Δb
- C. Update $\mathbf{w}$, b :
 $\mathbf{w} := \mathbf{w} + \Delta \mathbf{w}$, $b := b + \Delta b$

General Learning Principle

Let $\mathcal{D} = (\langle \mathbf{x}^{[1]}, y^{[1]} \rangle, \langle \mathbf{x}^{[2]}, y^{[2]} \rangle, \dots, \langle \mathbf{x}^{[n]}, y^{[n]} \rangle) \in (\mathbb{R}^m \times \{0, 1\})^n$

Most commonly used in DL, because

Minibatch mode

(mix between on-line and batch)

. Initialize $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m$, $\mathbf{b} := 0$

. For every training epoch:

A. Initialize $\Delta \mathbf{w} := 0$, $\Delta b := 0$

B. For every $\{\langle \mathbf{x}^{[i]}, y^{[i]} \rangle, \dots, \langle \mathbf{x}^{[i+k]}, y^{[i+k]} \rangle\} \subset \mathcal{D}$

(a) Compute output (prediction)

(b) Calculate error

(c) Update $\Delta \mathbf{w}$, Δb

C. Update $\mathbf{w}$, b :

$$\mathbf{w} := \mathbf{w} + \Delta \mathbf{w}, b := b + \Delta b$$

1. Choosing a subset (vs 1 example at a time) takes advantage of vectorization (faster iteration through epoch than on-line)
2. having fewer updates than "on-line" makes updates less noisy
3. makes more updates/epoch than "batch" and is thus faster

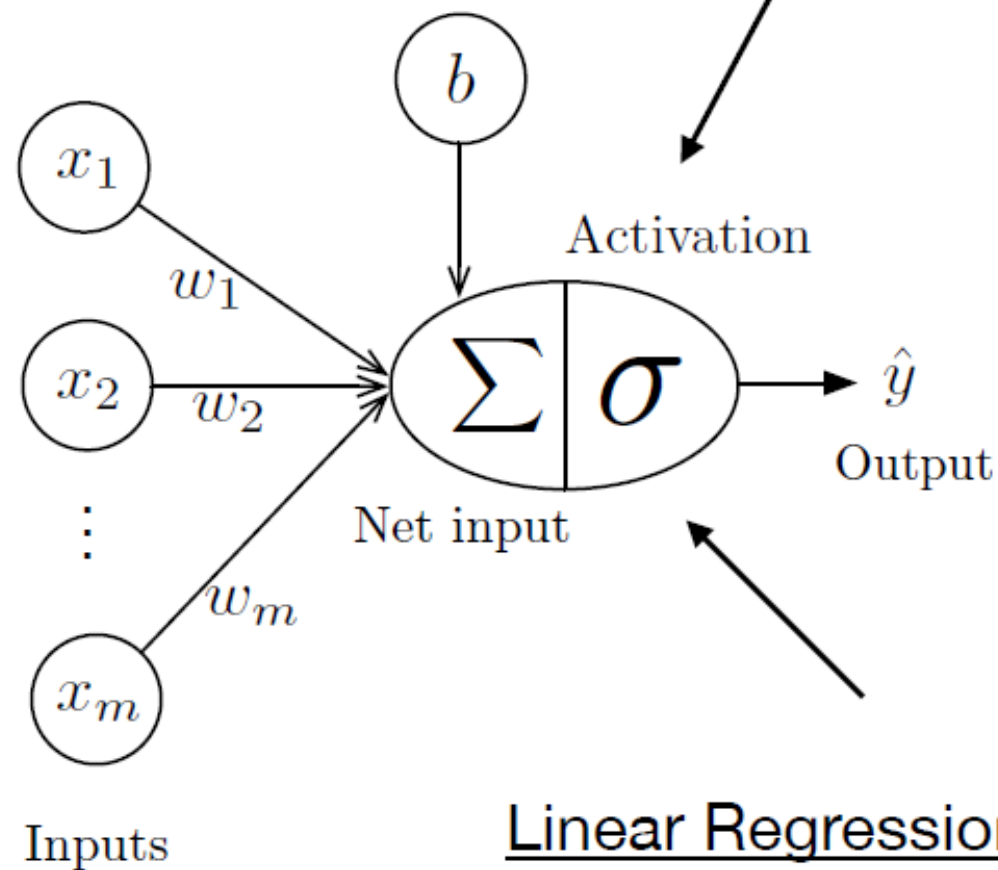
Linear regression as a single-layer neural network

1. Online, batch, and minibatch mode
- 2. Relation between perceptron and linear regression**
3. An iterative training algorithm for linear regression
4. Understanding gradient descent
5. Training an adaptive linear neuron (Adaline)
6. PyTorch Resources
7. Computation Graphs
8. Automatic Differentiation in PyTorch
9. Training ADALINE Manually Vs Automatically in PyTorch
10. A Closer Look at the PyTorch API

Linear Regression

Perceptron: Activation function is the threshold function

The output is a binary label $\hat{y} \in \{0, 1\}$



Linear Regression: Activation function is the identity function

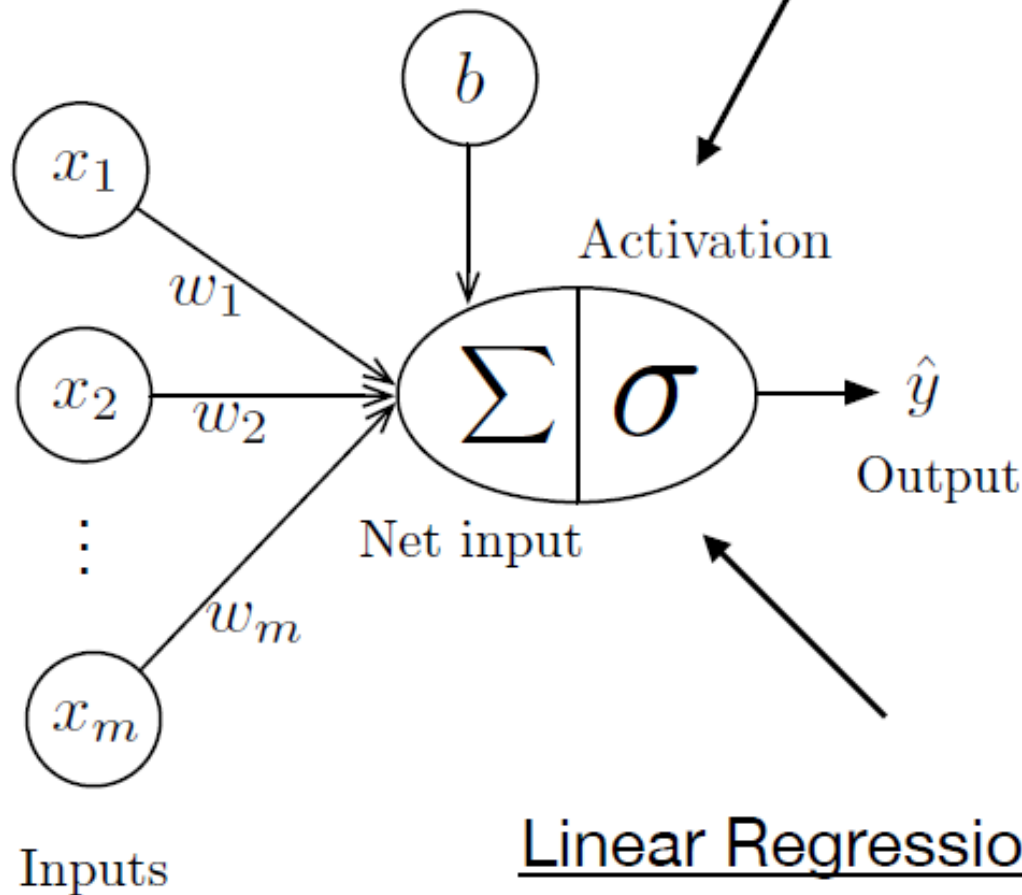
$$\sigma(x) = x$$

The output is a real number $\hat{y} \in \mathbb{R}$

Linear Regression

Perceptron: Activation function is the threshold function

The output is a binary label $\hat{y} \in \{0, 1\}$



You can think of linear regression as a linear neuron!

Linear Regression: Activation function is the identity function

$$\sigma(x) = x$$

The output is a real number $\hat{y} \in \mathbb{R}$

(Least-Squares) Linear Regression

In earlier statistics classes, you probably fit a linear regression model like this, using the "normal equations" (analytical solution):

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad \text{(assuming that the bias is included in } \mathbf{w}, \text{ and the design matrix has an additional vector of 1's)}$$

- Generally, this is the best approach for linear regression (although, the matrix inversion might be problematic on large datasets)
- However, we will now learn about another way to learn these parameters iteratively
- Why? Because this is what we will be doing in deep neural nets later, where we have large datasets, many connections, and non-convex loss functions

Training linear regression in an iterative fashion

1. Online, batch, and minibatch mode
2. Relation between perceptron and linear regression
- 3. An iterative training algorithm for linear regression**
4. Understanding gradient descent
5. Training an adaptive linear neuron (Adaline)
6. PyTorch Resources
7. Computation Graphs
8. Automatic Differentiation in PyTorch
9. Training ADALINE Manually Vs Automatically in PyTorch
10. A Closer Look at the PyTorch API

(Least-Squares) Linear Regression

-- iteratively with "brute force"

- A very naive way to fit a linear regression model (and any neural net) is to start with initializing the parameters to 0's or small random values
- Then, for k rounds
 - Choose another random set of weights
 - If the model performs better, keep those weights
 - If the model performs worse, discard the weights

This approach is guaranteed to find the optimal solution for very large k , but it would be terribly slow.

(Least-Squares) Linear Regression iteratively

- A very naive way to fit a linear regression model (and any neural net) is to start with initializing the parameters to 0's or small random values
- Then, for k rounds
 - Choose another random set of weights
 - If the model performs better, keep those weights
 - If the model performs worse, discard the weights

There's a better way!

- We will analyze what effect a change of a parameter has on the predictive performance (loss) of the model
then, we change the weight a little bit in the direction that improves the performance (minimizes the loss) the most
- We do this in several (small) steps until the loss does not further decrease

(Least-Squares) Linear Regression

The update rule turns out to be this:

"On-line" mode

Perceptron learning rule

1. Initialize $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m$, $b := 0$
2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$
 - (a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$
 - (b) $err := (y^{[i]} - \hat{y}^{[i]})$
 - (c) $\mathbf{w} := \mathbf{w} + err \times \mathbf{x}^{[i]}$
 $b := b + err$

Stochastic gradient descent

1. Initialize
2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$
 - (a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$
 - (b) $\nabla_{\mathbf{w}} \mathcal{L} = (y^{[i]} - \hat{y}^{[i]}) \mathbf{x}^{[i]}$
 $\nabla_b \mathcal{L} = (y^{[i]} - \hat{y}^{[i]})$
 - (c) $\mathbf{w} := \mathbf{w} + \eta \times (-\nabla_{\mathbf{w}} \mathcal{L})$
 $b := b + \eta \times \underbrace{(-\nabla_b \mathcal{L})}_{\substack{\uparrow \\ \text{negative gradient}}}$

$\nearrow$
learning rate

$\uparrow$
negative gradient

(Least-Squares) Linear Regression

The update rule turns out to be this:

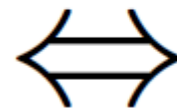
"On-line" mode:

Vectorized

1. Initialize
2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$
 - (a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$
 - (b) $\nabla_{\mathbf{w}} \mathcal{L} = (y^{[i]} - \hat{y}^{[i]}) \mathbf{x}^{[i]}$
 $\nabla_b \mathcal{L} = (y^{[i]} - \hat{y}^{[i]})$
 - (c) $\mathbf{w} := \mathbf{w} + \eta \times (-\nabla_{\mathbf{w}} \mathcal{L})$
 $b := b + \eta \times (-\nabla_b \mathcal{L})$

learning rate

negative gradient



For-Loop

1. Initialize
2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$
 - (a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$
 - B. For weight j in $\{1, \dots, m\}$:
 - (b) $\frac{\partial \mathcal{L}}{\partial w_j} = (y^{[i]} - \hat{y}^{[i]}) x_j^{[i]}$
 - (c) $w_j := w_j + \eta \times (-\frac{\partial \mathcal{L}}{\partial w_j})$
 - C.
 - $b := b + \eta \times (-\frac{\partial \mathcal{L}}{\partial b})$

(Least-Squares) Linear Regression

The update rule turns out to be this:

"On-line" mode

1. Initialize
2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$

(a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$

- B. For weight j in $\{1, \dots, m\}$:

(b) $\frac{\partial \mathcal{L}}{\partial w_j} = (y^{[i]} - \hat{y}^{[i]}) x_j^{[i]}$

(c) $w_j := w_j + \eta \times \left(-\frac{\partial \mathcal{L}}{\partial w_j}\right)$

C.

$$b := b + \eta \times \left(-\frac{\partial \mathcal{L}}{\partial b}\right)$$

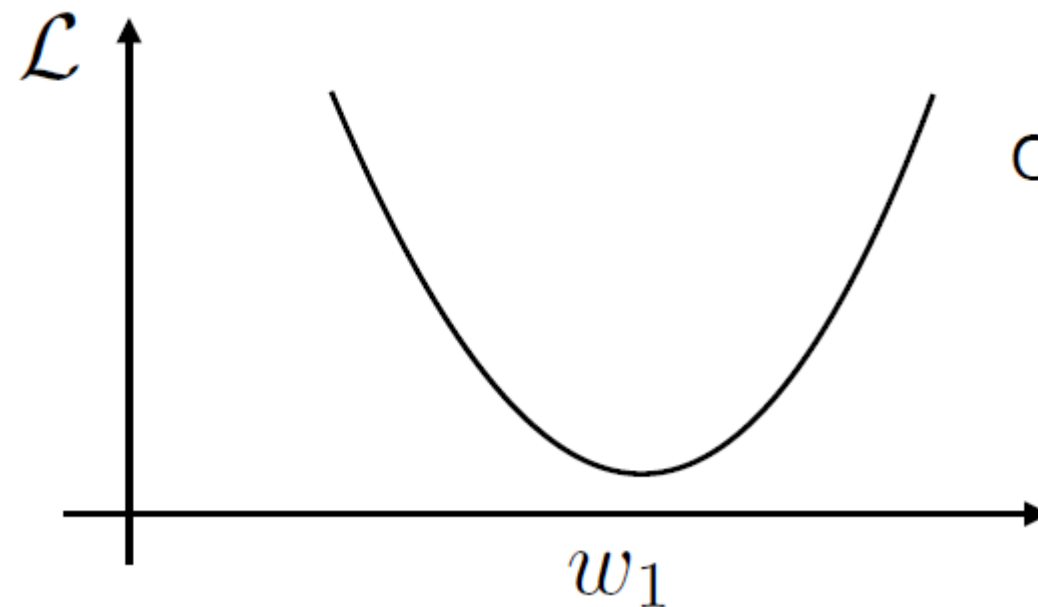
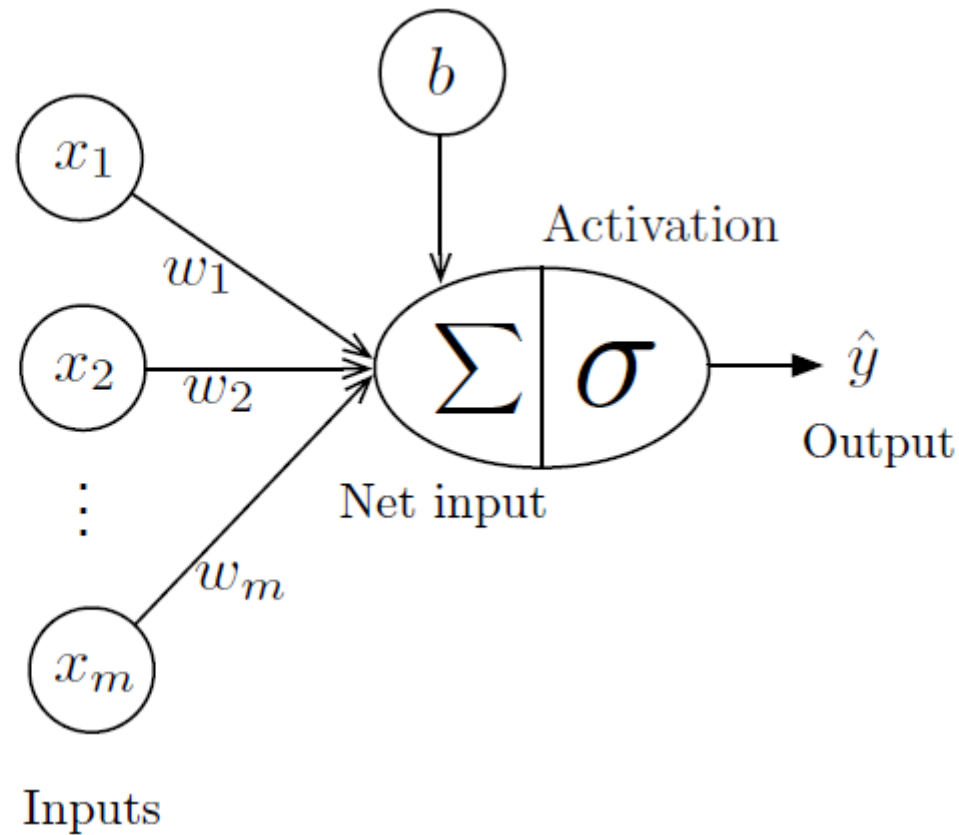
Coincidentally, this appears almost to be the same as the perceptron rule, except that the prediction is a real number and we have a learning rate

This learning rule (from the previous slide)
is called (stochastic) gradient descent.
So, how did we get there?

Training linear regression with gradient descent

1. Online, batch, and minibatch mode
2. Relation between perceptron and linear regression
3. An iterative training algorithm for linear regression
- 4. Understanding gradient descent**
5. Training an adaptive linear neuron (Adaline)
6. PyTorch Resources
7. Computation Graphs
8. Automatic Differentiation in PyTorch
9. Training ADALINE Manually Vs Automatically in PyTorch
10. A Closer Look at the PyTorch API

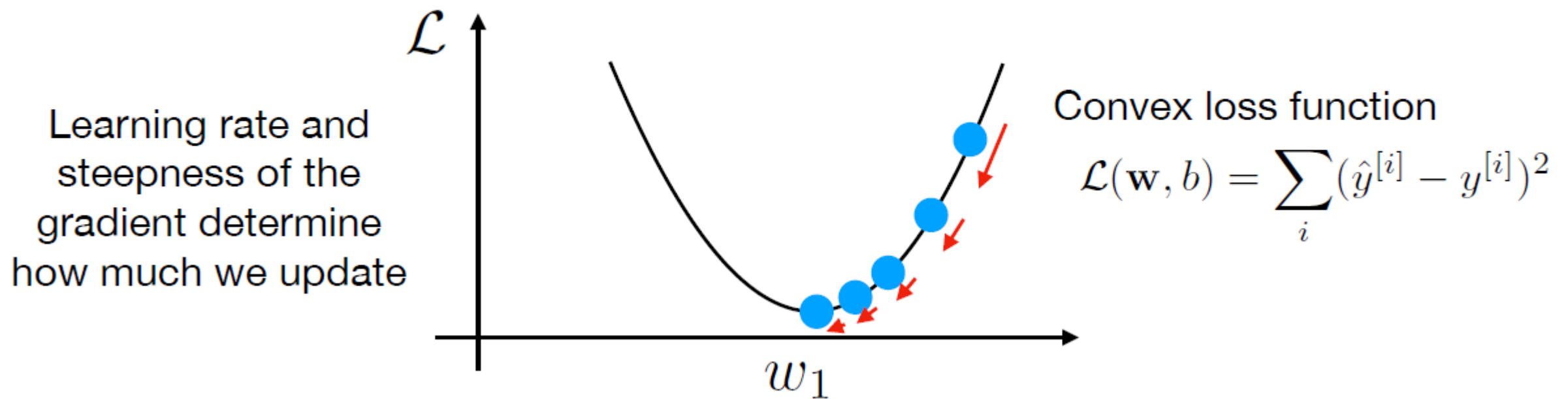
Back to Linear Regression



Convex loss function

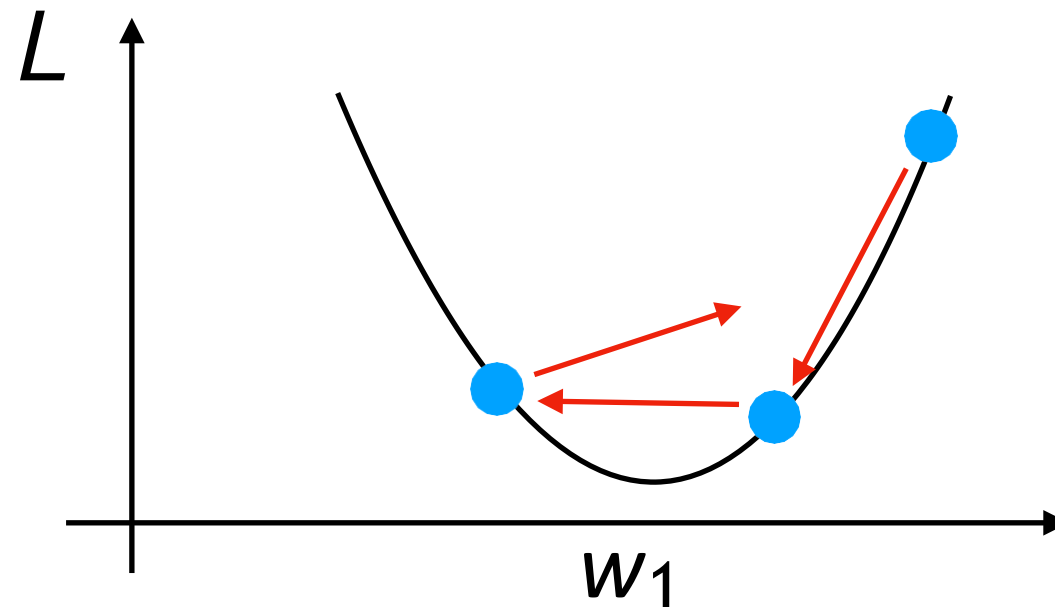
$$\mathcal{L}(\mathbf{w}, b) = \sum_i (\hat{y}^{[i]} - y^{[i]})^2$$

Gradient Descent

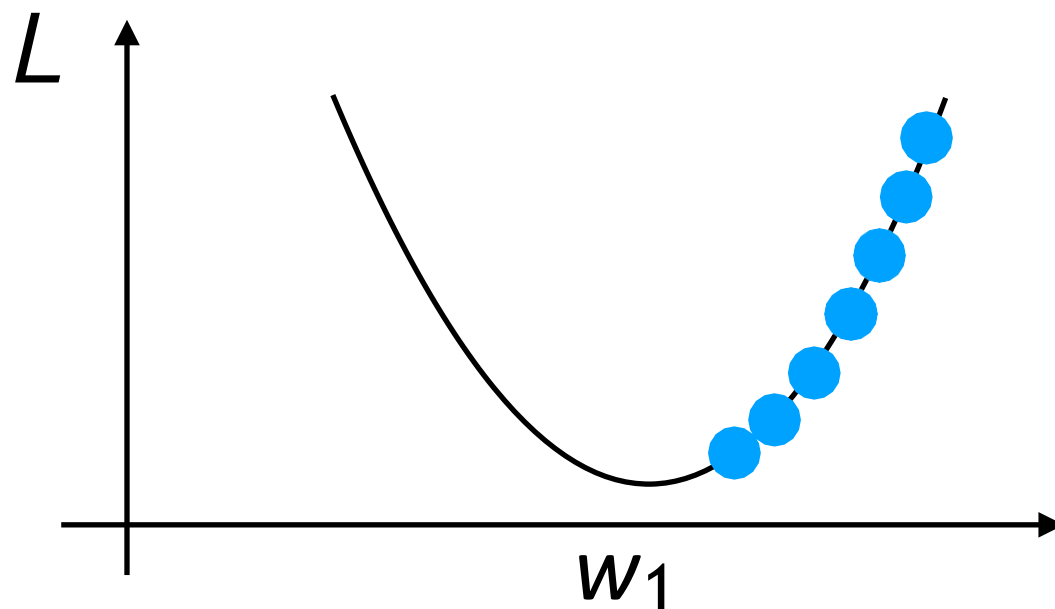


Gradient Descent

If the learning rate is too large,
we can overshoot



If the learning rate is too small,
convergence is very slow



(Least-Squares) Linear Regression

The update rule turns out to be this:

"On-line" mode

Perceptron learning rule

1. Initialize $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m$, $b := 0$
2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$
 - (a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$
 - (b) $err := (y^{[i]} - \hat{y}^{[i]})$
 - (c) $\mathbf{w} := \mathbf{w} + err \times \mathbf{x}^{[i]}$
 $b := b + err$

Stochastic gradient descent

1. Initialize
2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$
 - (a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$
 - (b) $\nabla_{\mathbf{w}} \mathcal{L} = (y^{[i]} - \hat{y}^{[i]}) \mathbf{x}^{[i]}$
 $\nabla_b \mathcal{L} = (y^{[i]} - \hat{y}^{[i]})$
 - (c) $\mathbf{w} := \mathbf{w} + \eta \times (-\nabla_{\mathbf{w}} \mathcal{L})$
 $b := b + \eta \times \underbrace{(-\nabla_b \mathcal{L})}_{\substack{\uparrow \\ \text{negative gradient}}}$

$\nearrow$
learning rate

$\uparrow$
negative gradient

Linear Regression Loss Derivative

$$\mathcal{L}(\mathbf{w}, b) = \sum_i (\hat{y}^{[i]} - y^{[i]})^2 \quad \text{Sum Squared Error (SSE) loss}$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_j} &= \frac{\partial}{\partial w_j} \sum_i (\hat{y}^{[i]} - y^{[i]})^2 \\ &= \frac{\partial}{\partial w_j} \sum_i (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]})^2 \\ &= \sum_i 2(\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) \frac{\partial}{\partial w_j} (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) \\ &= \sum_i 2(\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) \frac{d\sigma}{d(\mathbf{w}^T \mathbf{x}^{[i]})} \frac{\partial}{\partial w_j} \mathbf{w}^T \mathbf{x}^{[i]} \\ &= \sum_i 2(\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) \frac{d\sigma}{d(\mathbf{w}^T \mathbf{x}^{[i]})} x_j^{[i]} \quad \text{(Note that the activation function is the identity function in linear regression)} \\ &= \sum_i 2(\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) x_j^{[i]} \end{aligned}$$

Linear Regression Loss Derivative (alt.)

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{2n} \sum_i (\hat{y}^{[i]} - y^{[i]})^2$$

Mean Squared Error (MSE) loss often scaled by factor 1/2 for convenience

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_j} &= \frac{\partial}{\partial w_j} \frac{1}{2n} \sum_i (\hat{y}^{[i]} - y^{[i]})^2 \\ &= \frac{\partial}{\partial w_j} \sum_i \frac{1}{2n} (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]})^2 \end{aligned}$$

$$= \sum_i \frac{1}{n} (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) \frac{\partial}{\partial w_j} (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]})$$

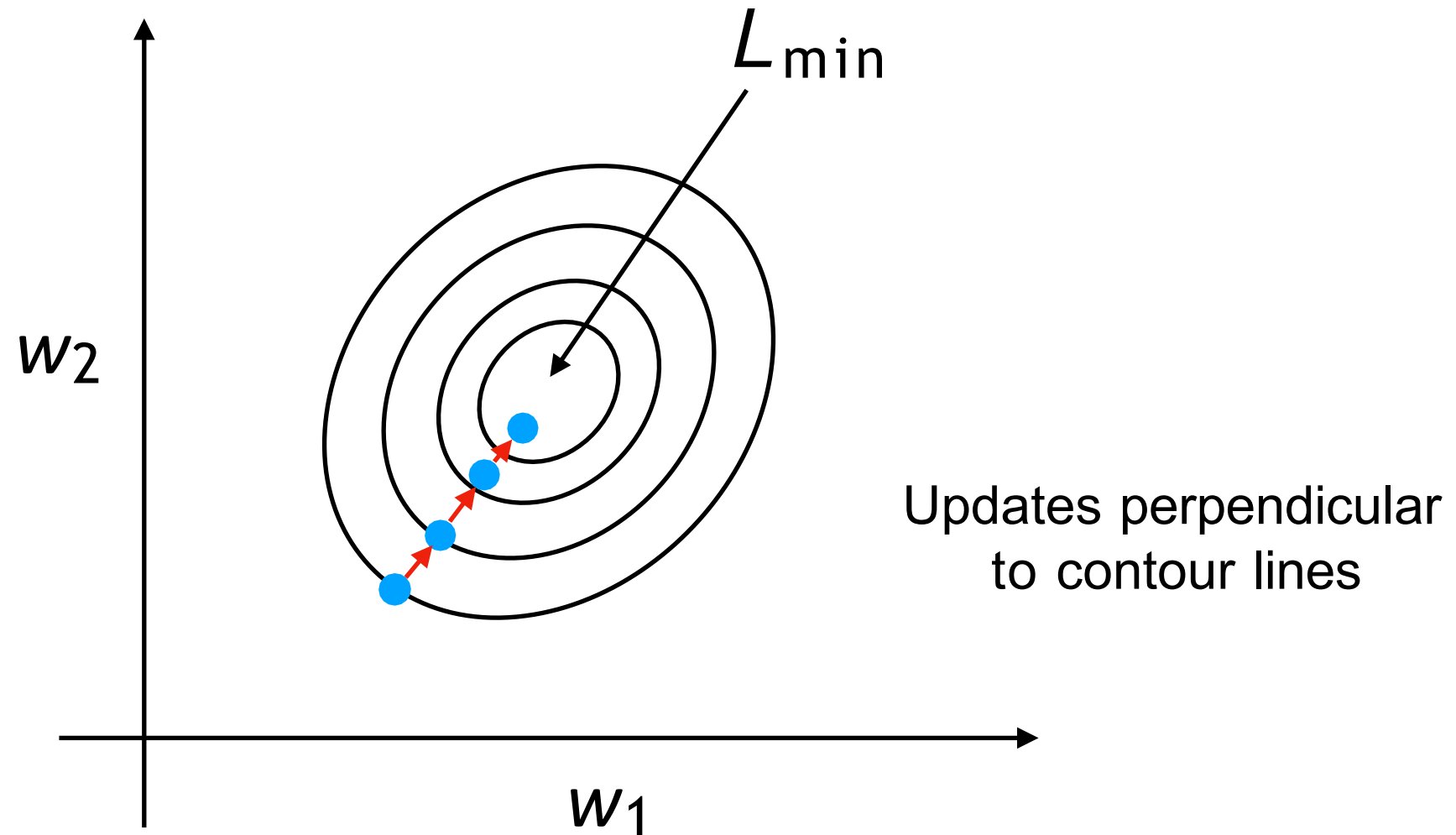
$$= \frac{1}{n} \sum_i (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) \frac{d\sigma}{d(\mathbf{w}^T \mathbf{x}^{[i]})} \frac{\partial}{\partial w_j} \mathbf{w}^T \mathbf{x}^{[i]}$$

$$= \frac{1}{n} \sum_i (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) \frac{d\sigma}{d(\mathbf{w}^T \mathbf{x}^{[i]})} x_j^{[i]}$$

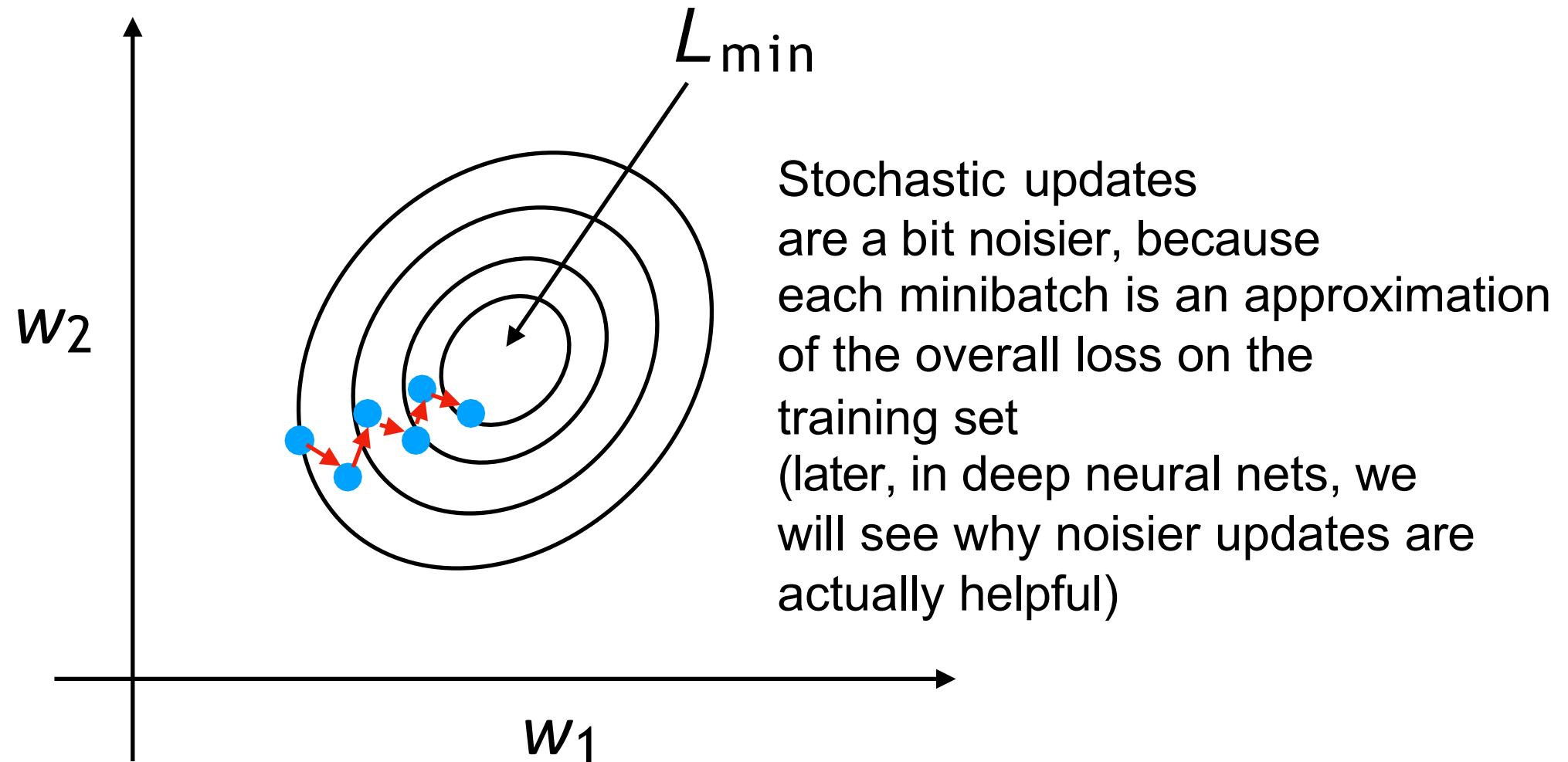
(Note that the activation function is the identity function in linear regression)

$$= \frac{1}{n} \sum_i (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) x_j^{[i]}$$

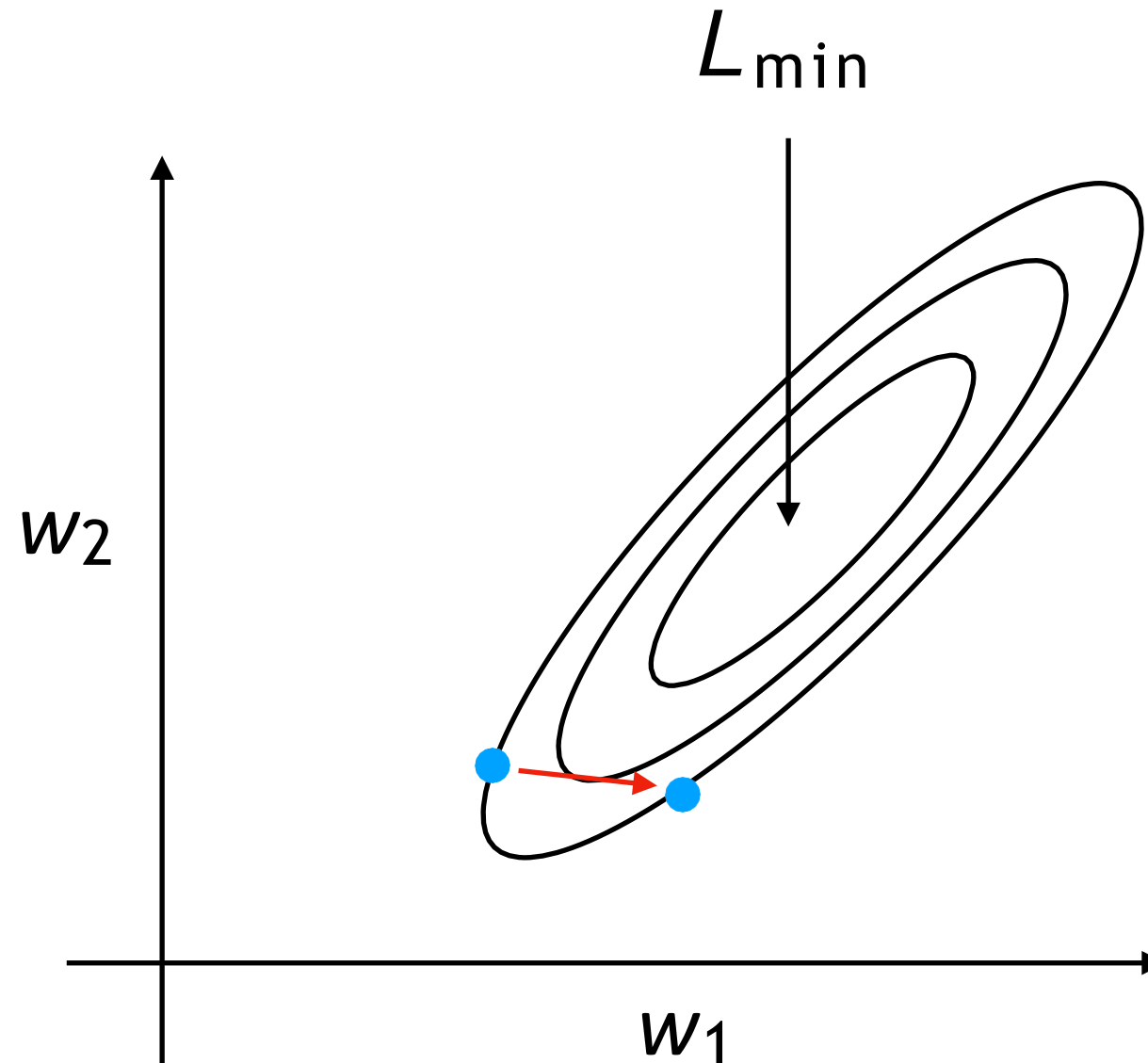
Batch Gradient Descent as Surface Plot



Stochastic Gradient Descent as Surface Plot



Batch Gradient Descent as Surface Plot



If inputs are on very different scales
some weights will update more than
others ... and it will also harm convergence
(always normalize inputs!)

Training a single-layer neural network with gradient descent

1. Online, batch, and minibatch mode
2. Relation between perceptron and linear regression
3. An iterative training algorithm for linear regression
4. Understanding gradient descent
- 5. Training an adaptive linear neuron (Adaline)**
6. PyTorch Resources
7. Computation Graphs
8. Automatic Differentiation in PyTorch
9. Training ADALINE Manually Vs Automatically in PyTorch
10. A Closer Look at the PyTorch API

ADALINE

Widrow and Hoff's ADALINE (1960)

A nicely differentiable neuron model

Widrow, B., & Hoff, M. E. (1960). *Adaptive switching circuits* (No. TR-1553-1). Stanford Univ Ca Stanford Electronics Labs.

Widrow, B. (1960). *Adaptive "adaline" Neuron Using Chemical" memistors."*

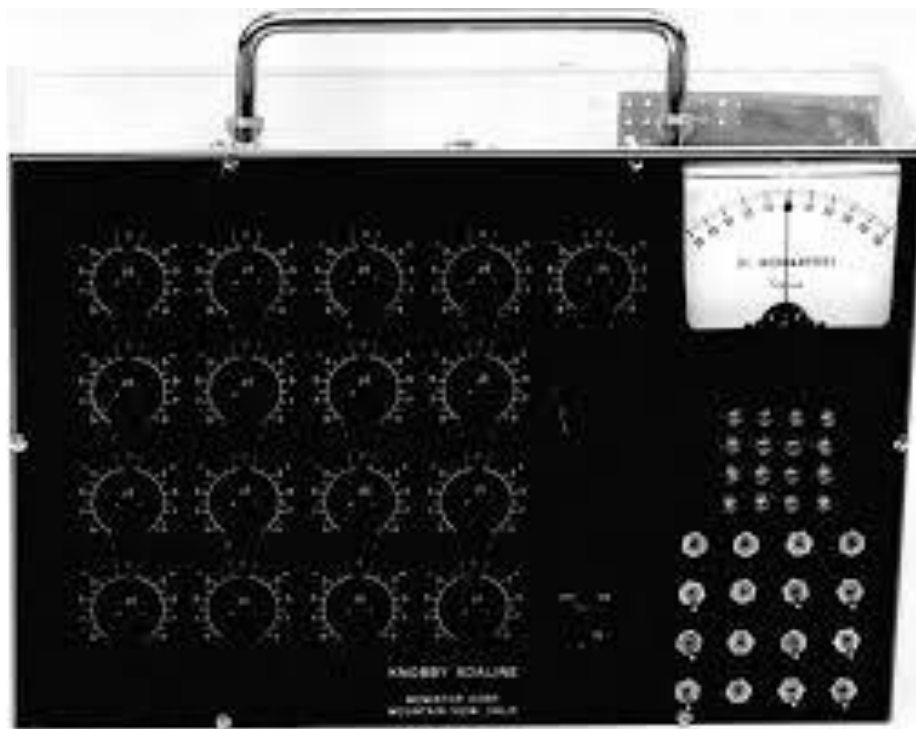
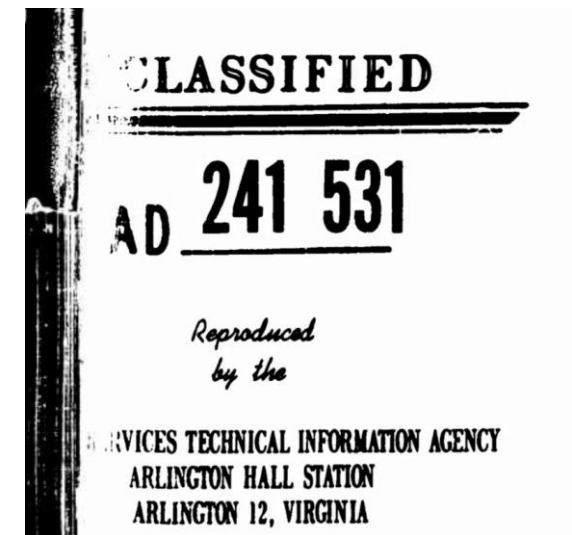


Image source: https://www.researchgate.net/profile/Alexander_Magoun2/publication/265789430/figure/fig2/AS:392335251787780@1470551421849/ADALINE-An-adaptive-linear-neuron-Manually-adapted-synapses-Designed-and-built-by-Ted.png



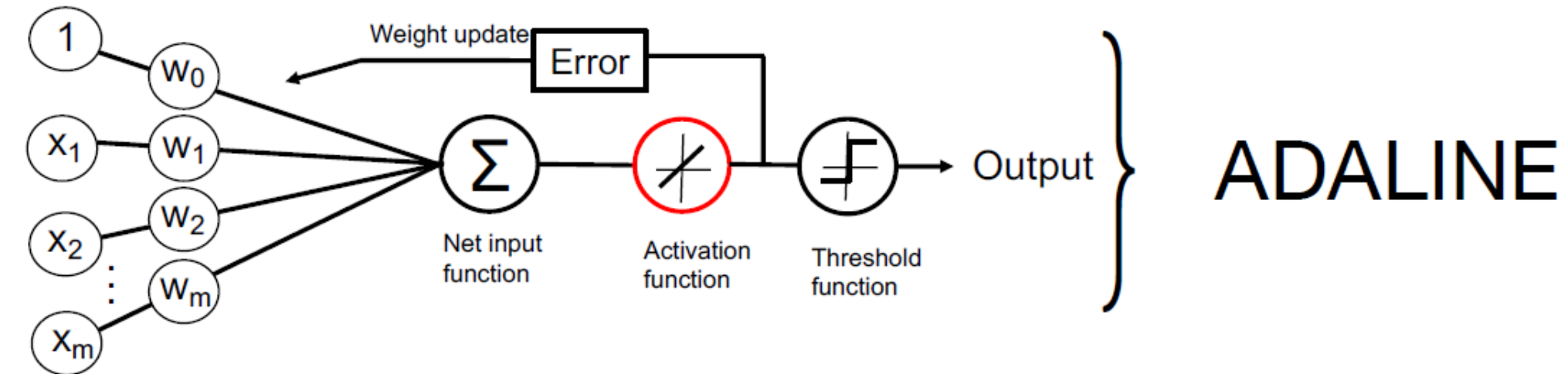
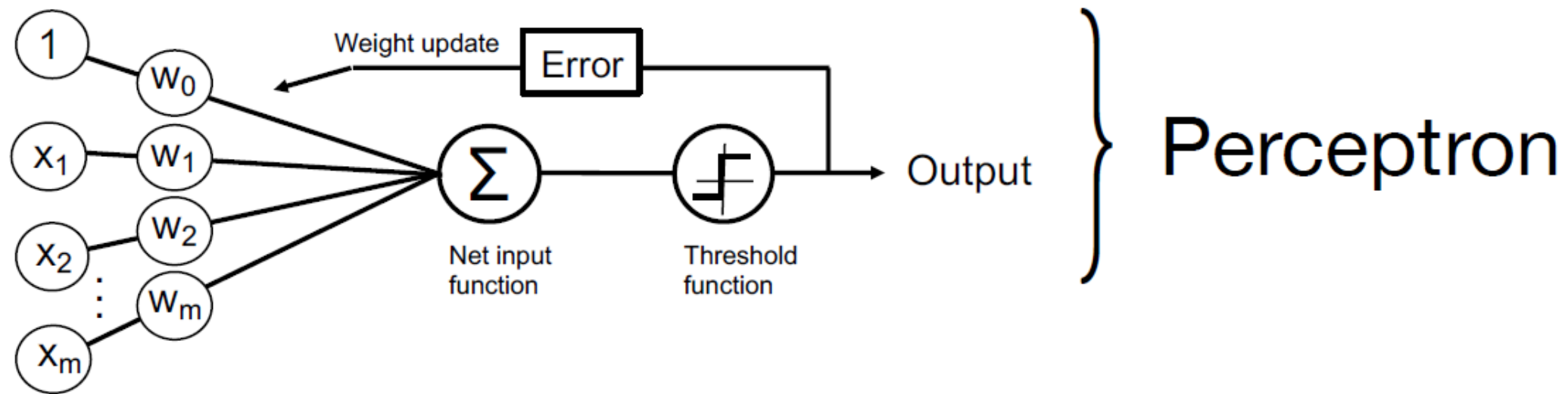
THIS REPORT HAS BEEN DELIMITED
AND CLEARED FOR PUBLIC RELEASE
UNDER DOD DIRECTIVE 5200.20 AND
NO RESTRICTIONS ARE IMPOSED UPON
ITS USE AND DISCLOSURE.

DISTRIBUTION STATEMENT A

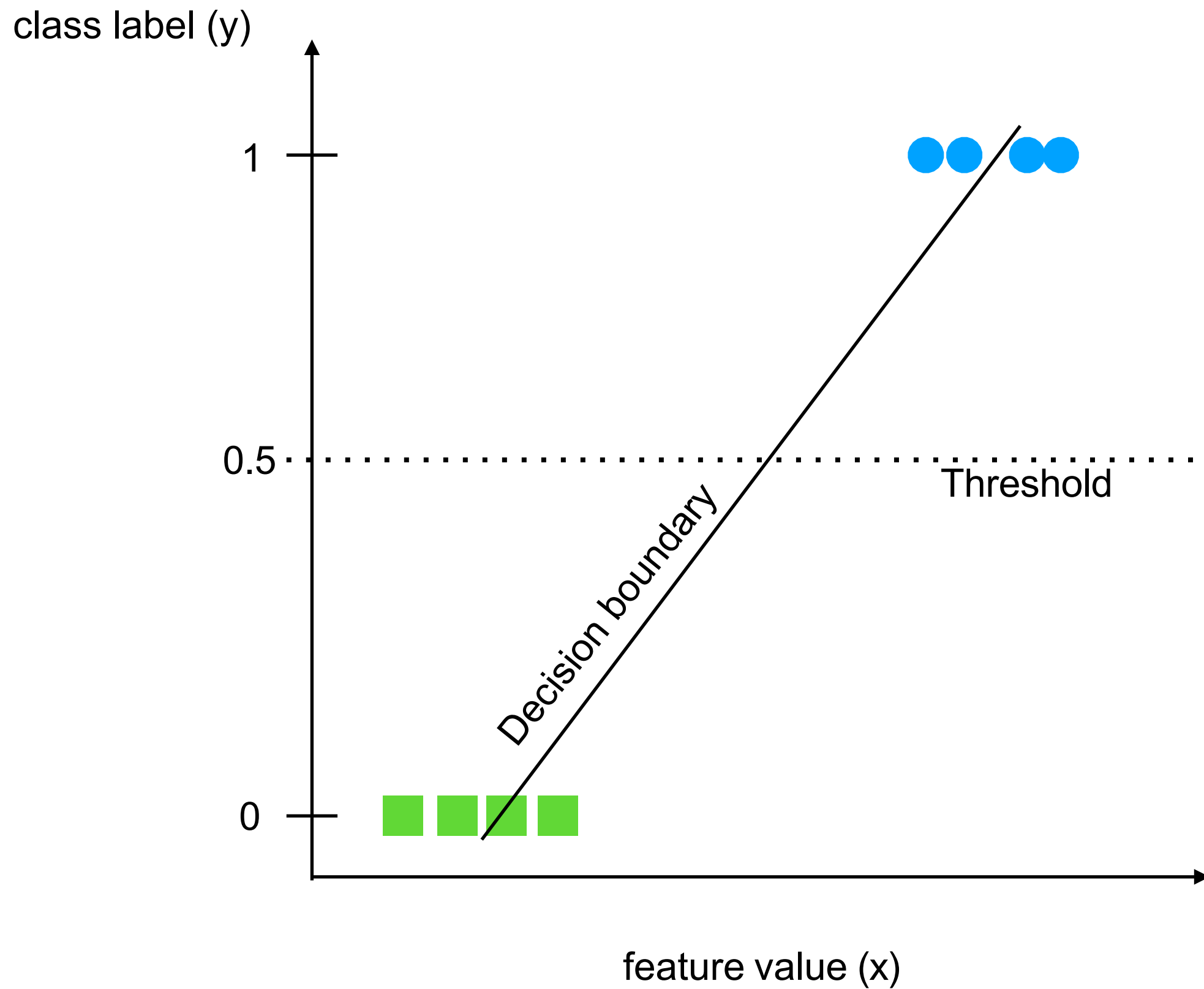
APPROVED FOR PUBLIC RELEASE;
DISTRIBUTION UNLIMITED.

ADALINE

ADaptive LInear NEuron



Linear Regression



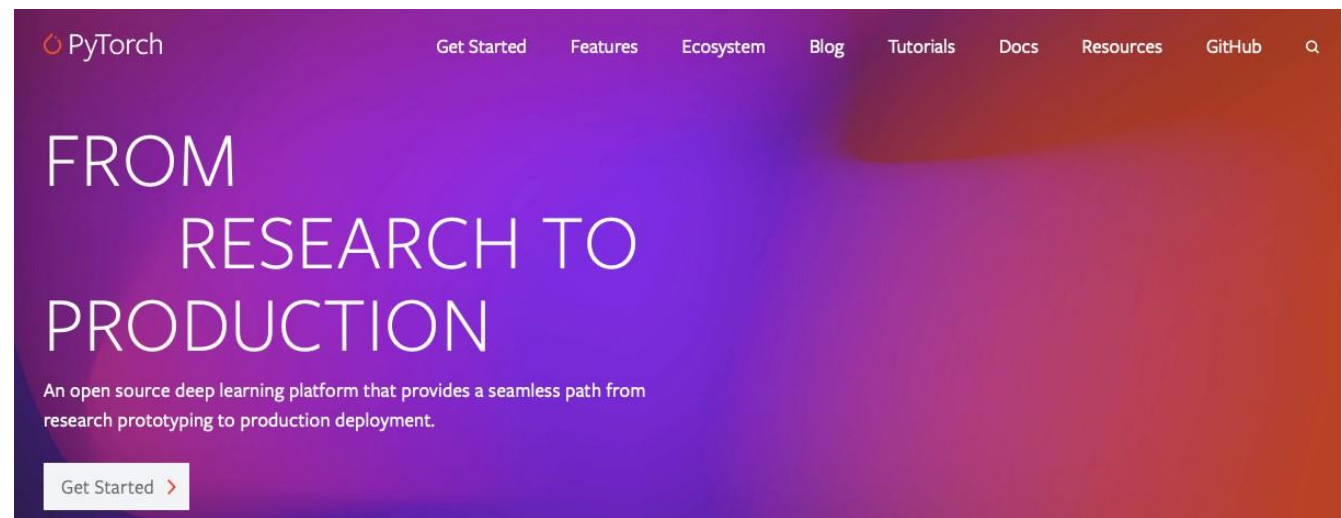
Computing partial derivatives more easily (and automatically) with PyTorch

Lecture Overview

1. Online, batch, and minibatch mode
2. Relation between perceptron and linear regression
3. An iterative training algorithm for linear regression
4. Understanding gradient descent
5. Training an adaptive linear neuron (Adaline)
6. **PyTorch Resources**
7. **Computation Graphs**
8. **Automatic Differentiation in PyTorch**
9. **Training ADALINE Manually Vs Automatically in PyTorch**
10. **A Closer Look at the PyTorch API**

Learning More About PyTorch

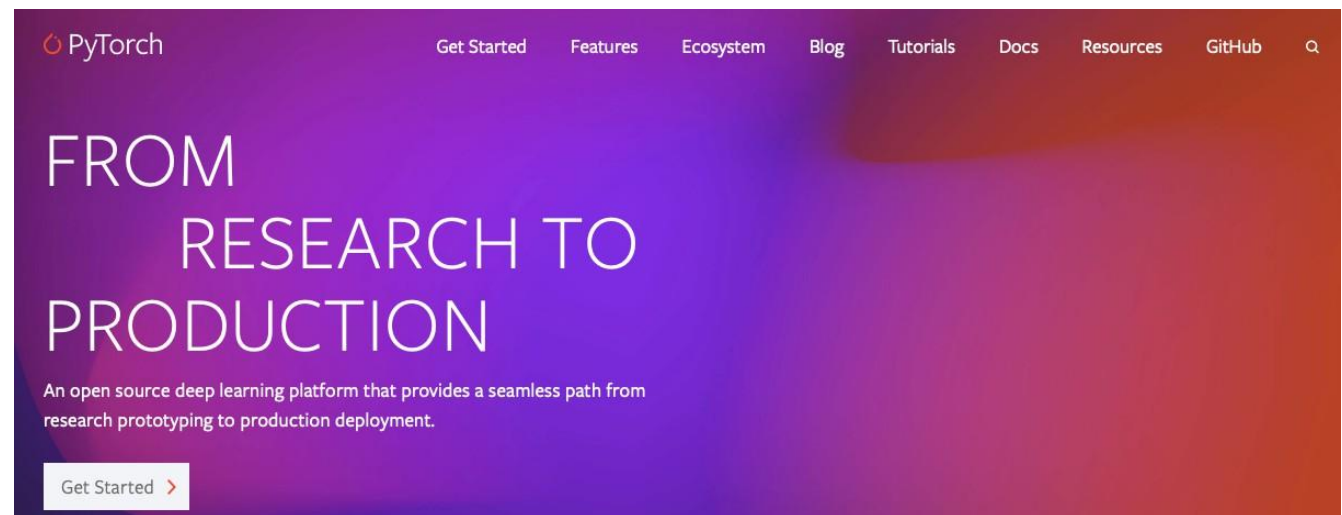
1. Online, batch, and minibatch mode
2. Relation between perceptron and linear regression
3. An iterative training algorithm for linear regression
4. Understanding gradient descent
5. Training an adaptive linear neuron (Adaline)
- 6. PyTorch Resources**
7. Computation Graphs
8. Automatic Differentiation in PyTorch
9. Training ADALINE Manually Vs Automatically in PyTorch
10. A Closer Look at the PyTorch API



<https://pytorch.org/>

At a Glance:

- **PyTorch started in 2016**
- **Focuses on flexibility and minimizing cognitive overhead**
- **Dynamic nature of autograd API inspired by Chainer**
- **Core features**
 - Automatic differentiation
 - Dynamic computation graphs
 - NumPy integration
- **written in C++ and CUDA (CUDA is like C++ for the GPU)**
- **Python is the usability glue**



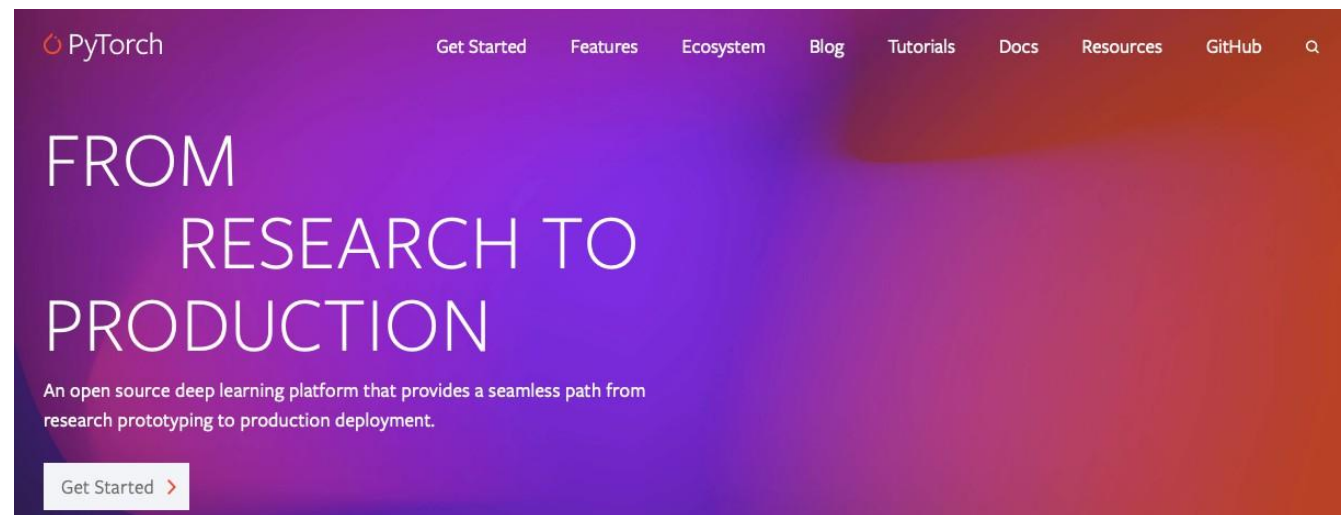
<https://pytorch.org/>

At a Glance:

- PyTorch started in 2016
- Focuses on flexibility and minimizing cognitive overhead
- Dynamic nature of autograd API inspired by Chainer
- Core features
 - Automatic differentiation
 - Dynamic computation graphs
 - NumPy integration
- written in C++ and CUDA (CUDA is not Python)
- Python is the usability glue

PyTorch vs NumPy

- Support GPU
- distribute ops across multiple devices
- keep track of computation graph and ops that created them



<https://pytorch.org/>

At a Glance:

- PyTorch started in 2016
- Focuses on flexibility and minimizing cognitive overhead
- Dynamic nature of autograd API inspired by Chainer
- Core features
 - Automatic differentiation

"the speedup gained by taking Python out of the computation is 10% or less"

-- Stevens *et al.*: Deep Learning with PyTorch

- written in C++ and CUDA (CUDA is like C++ for the GPU)
- Python is the usability glue

Installation

Recommendation for Laptop (e.g., Windows)

Recommendation for Desktop (Linux) with GPU

NOTE: Latest Stable PyTorch requires Python 3.10 or later.

PyTorch Build	Stable (2.14.0)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Pip	LibTorch	Source	
Language	Python		C++ / Java	
Compute Platform	CUDA 12.6	CUDA 13.0	CUDA 13.2	CPU
Run this Command:	pip3 install torch torchvision			

PyTorch Build	Stable (2.14.0)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Pip	LibTorch	Source	
Language	Python		C++ / Java	
Compute Platform	CUDA 12.6	CUDA 13.0	CUDA 13.2	CPU
Run this Command:	pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu132			

<https://pytorch.org/>

And don't forget that you import PyTorch as "import torch," not "import pytorch" :)

```
import torch
torch.__version__
```

✓ 0.0s

'2.14.0+cpu'

Many Useful Tutorials (recommend that you read some of them)

PyTorch

Get StartedEcosystemMobileBlogTutorialsDocsResourcesGithub

1.7.1

 Search Tutorials

PyTorch Recipes

See All Recipes

Learning PyTorch

Deep Learning with PyTorch: A 60 Minute Blitz

Learning PyTorch with Examples

What is *torch.nn* really?

Visualizing Models, Data, and Training with TensorBoard

Image/Video

TorchVision Object Detection Finetuning Tutorial

Transfer Learning for Computer Vision Tutorial

Adversarial Example Generation

Tutorials > Welcome to PyTorch Tutorials

 Shortcuts

WELCOME TO PYTORCH TUTORIALS

Welcome to PyTorch Tutorials

Additional Resources

New to PyTorch?

The 60 min blitz is the most common starting point and provides a broad view on how to use PyTorch. It covers the basics all the way to constructing deep neural networks.

Start 60-min blitz

PyTorch Recipes

Bite-size, ready-to-deploy PyTorch code examples.

Explore Recipes

AllAudioBest PracticeC++CUDAFrontend APIs

Getting StartedImage/VideoInterpretabilityMemory Format

<https://pytorch.org/tutorials/>

Many Useful Tutorials (recommend that you read some of them)

PyTorch

[Get Started](#) [Ecosystem](#) [Mobile](#) [Blog](#) [Tutorials](#) [Docs](#) [Resources](#) [Github](#)

1.7.1

 Search Tutorials

[PyTorch Recipes](#)
[See All Recipes](#)

[Learning PyTorch](#)
[Deep Learning with PyTorch: A 60 Minute Blitz](#)
[Learning PyTorch with Examples](#)
[What is torch.nn really?](#)
[Visualizing Models, Data, and Training with TensorBoard](#)

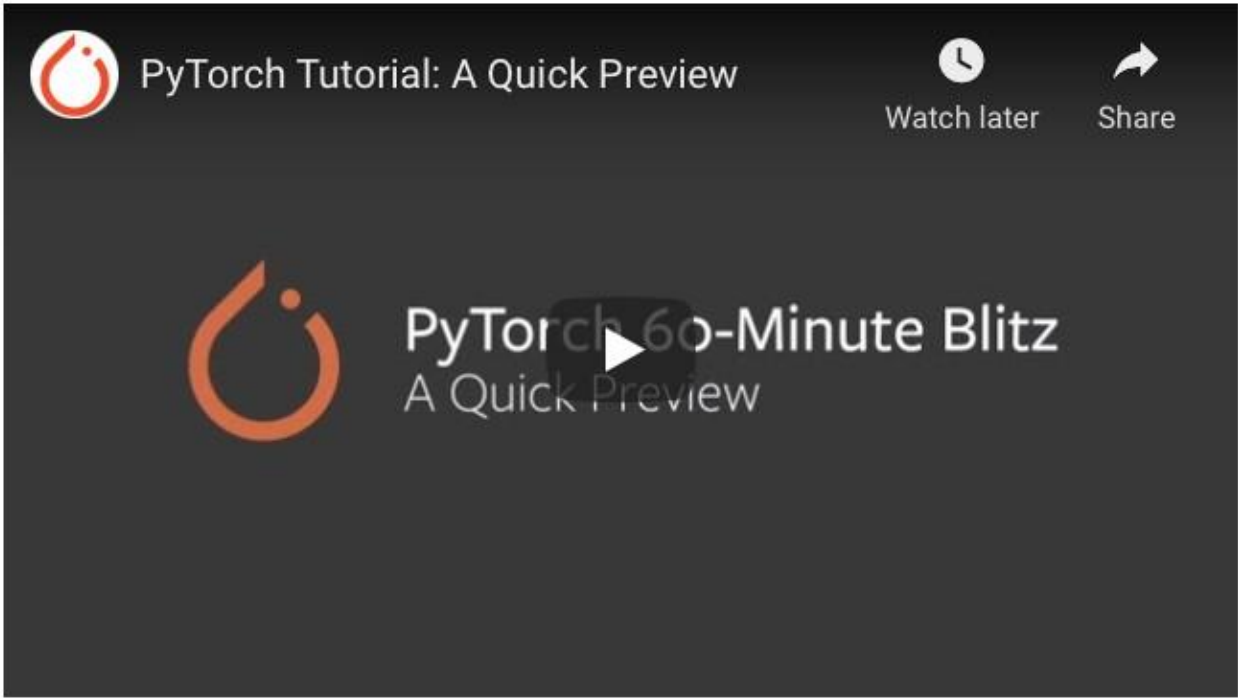
[Image/Video](#)
[TorchVision Object Detection Finetuning Tutorial](#)
[Transfer Learning for Computer Vision Tutorial](#)
[Adversarial Example Generation](#)
[DCGAN Tutorial](#)

[Tutorials](#) > Deep Learning with PyTorch: A 60 Minute Blitz [Shortcuts](#)

DEEP LEARNING WITH PYTORCH: A 60 MINUTE BLITZ

Author: Soumith Chintala

 PyTorch Tutorial: A Quick Preview Watch later Share



PyTorch 60-Minute Blitz
A Quick Preview

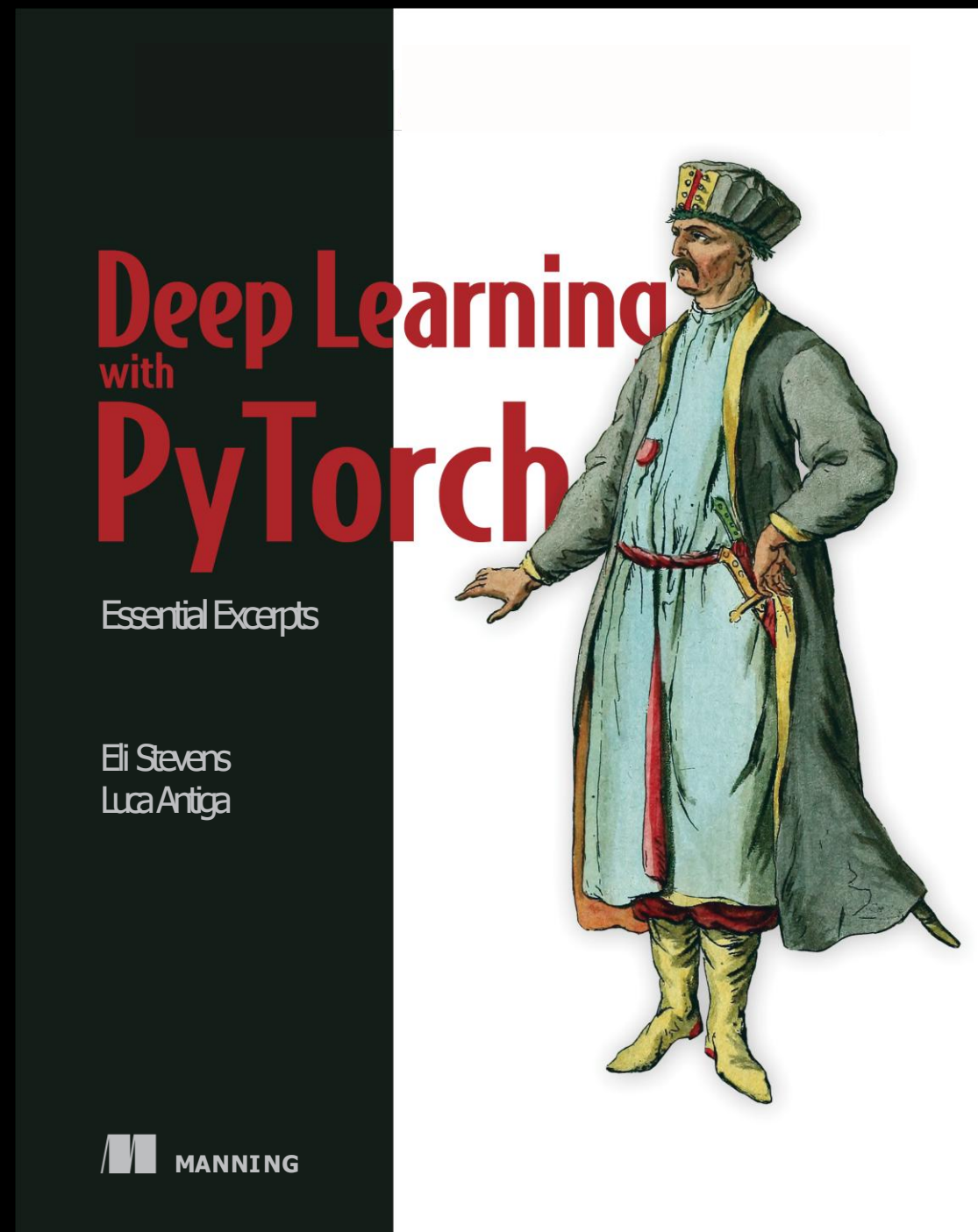
What is PyTorch?

Deep Learning with PyTorch
A 60 Minute Blitz

What is PyTorch?

Goal of this tutorial:

https://pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html



<https://pytorch.org/assets/deep-learning/Deep-Learning-with-PyTorch.pdf>

Very Active & Friendly Community and Help/Discussion Forum





Do you want live notifications when people reply to your posts? [Enable Notifications](#) 

all categories ▸

all ▾

Latest

New (47)

Unread (104)

Top

Categories

+ New Topic

Topic		Replies	Views	Activity
☒ Using MSELoss instead of CrossEntropy for Ordinal Regression/Classification ■ vision		2	83	1h
Optimizer.load_state_dict() weird behaviour with Adam optimizer ■ vision		7	2.0k	1h
Is there a way to train 3 dataloaders using multiprocessing? •		0	11	2h
☒ Getting different feature vectors from frozen layers after training ■ vision		5	86	2h
☒ Libtorch_cuda.so is too large (>2GB) ■ deployment		22	346	2h
Undo pruning - How to 'unmask' pruned weights •		0	8	2h
If input.dim() == 2 and bias is not None: AttributeError: 'tuple' object has no attribute 'dim' •		3	41	2h
Export unsupported/compound ops to ONNX • ■ deployment		0	9	2h

<https://discuss.pytorch.org>

Understanding Automatic Differentiation via Computation Graphs

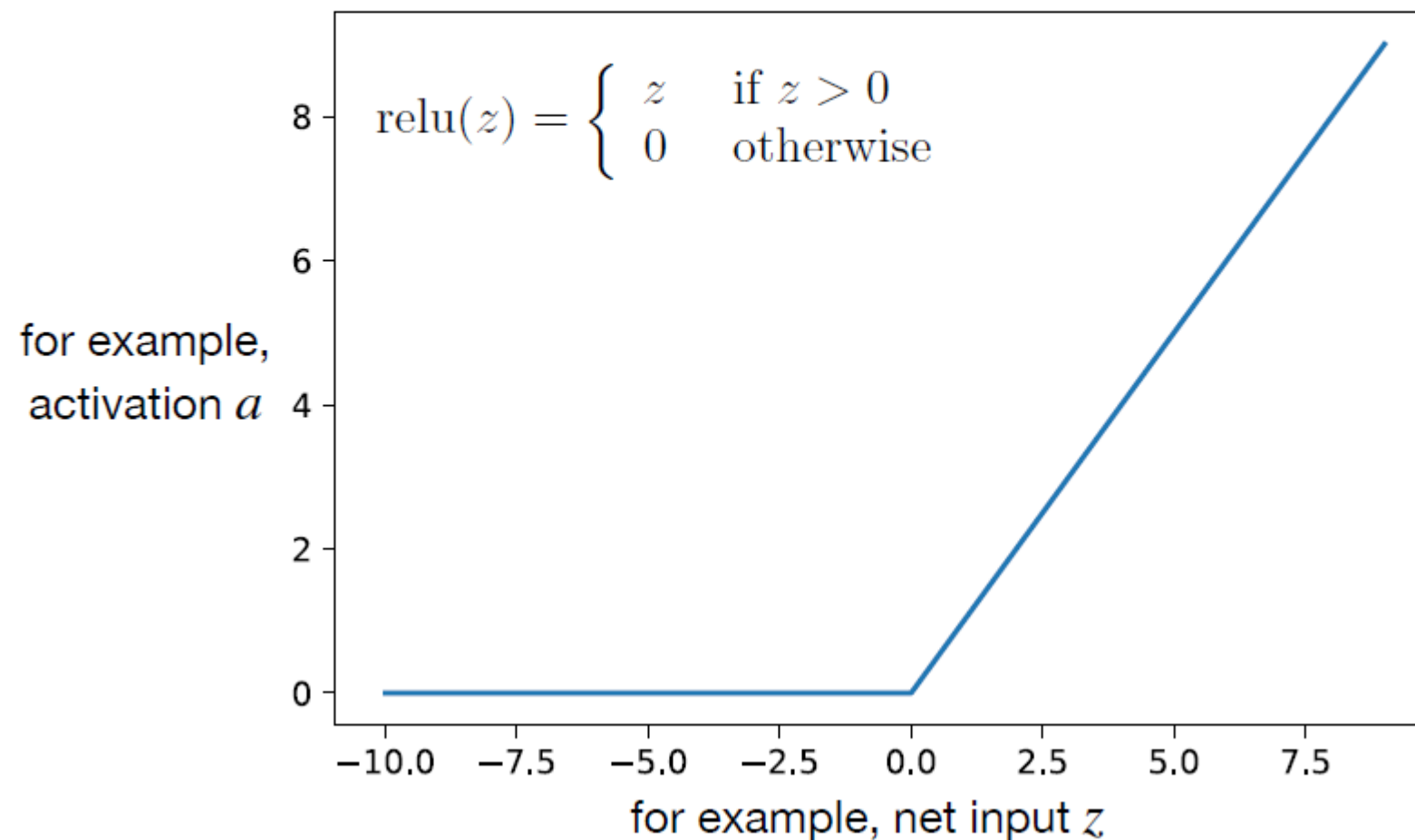
1. Online, batch, and minibatch mode
2. Relation between perceptron and linear regression
3. An iterative training algorithm for linear regression
4. Understanding gradient descent
5. Training an adaptive linear neuron (Adaline)
6. PyTorch Resources
- 7. Computation Graphs**
8. Automatic Differentiation in PyTorch
9. Training ADALINE Manually Vs Automatically in PyTorch
10. A Closer Look at the PyTorch API

In the context of deep learning (and PyTorch)
it is helpful to think about neural networks
as computation graphs

Computation Graphs

Suppose we have the following activation function:

$$a(x, w, b) = \text{relu}(w \cdot x + b)$$



ReLU = Rectified Linear Unit
(prob. the most commonly used activation function in DL)

Side-note about ReLU Function

You may note that

$$\sigma'(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z > 0 \\ \text{DNE} & \text{if } z = 0 \end{cases}$$

But in the machine learning--computer science context, for convenience, we can just say

$$\sigma'(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ 1 & \text{if } z > 0 \end{cases}$$

Why not differentiable?

Derivative does not exist (DNE) at 0, because the derivative is different if we approach the limit from the left or right:

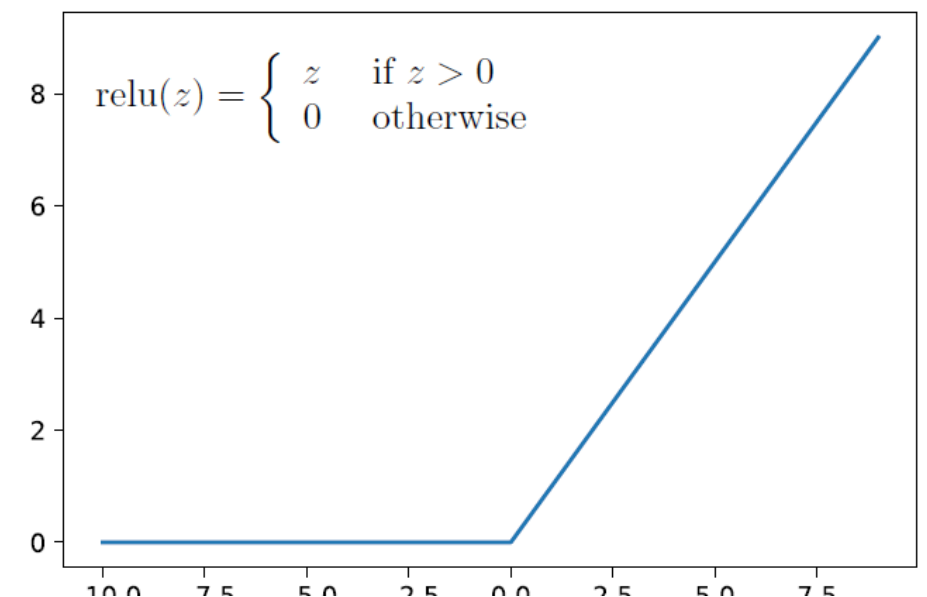
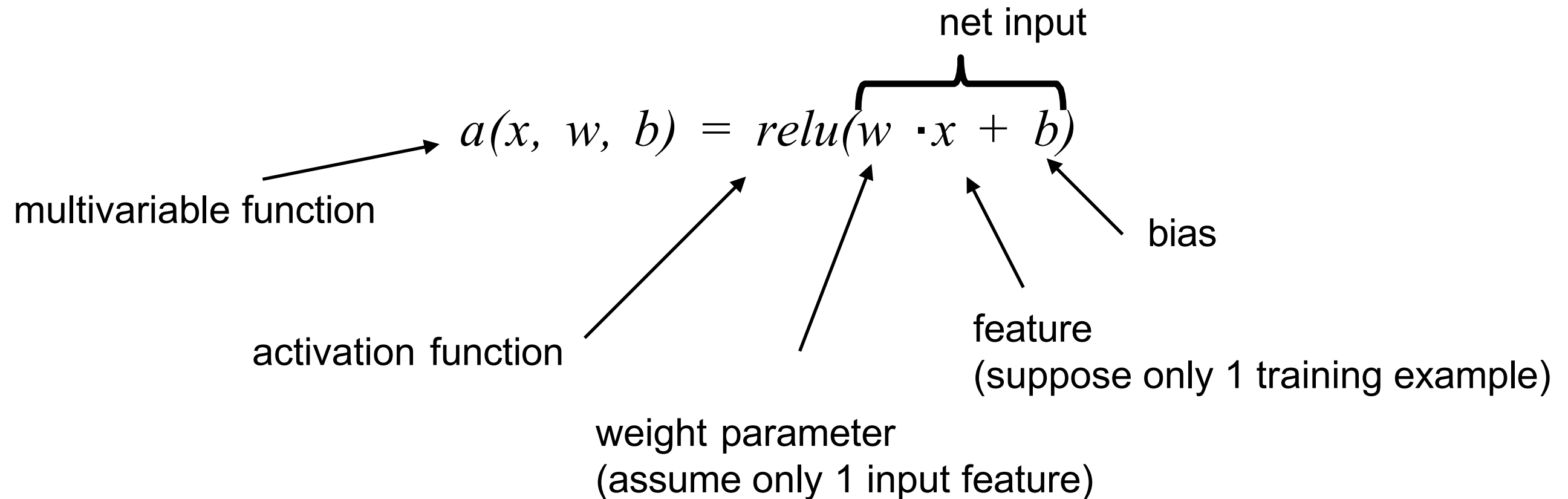
$$\sigma'(z) = \lim_{\Delta z \rightarrow 0} \frac{\max(0, z + \Delta z) - \max(0, z)}{\Delta z}$$

$$\sigma'(0) = \lim_{\Delta z \rightarrow 0^+} \frac{0 + \Delta z - 0}{\Delta z} = 1$$

$$\sigma'(0) = \lim_{\Delta z \rightarrow 0^-} \frac{0 - 0}{\Delta z} = 0$$

Computation Graphs

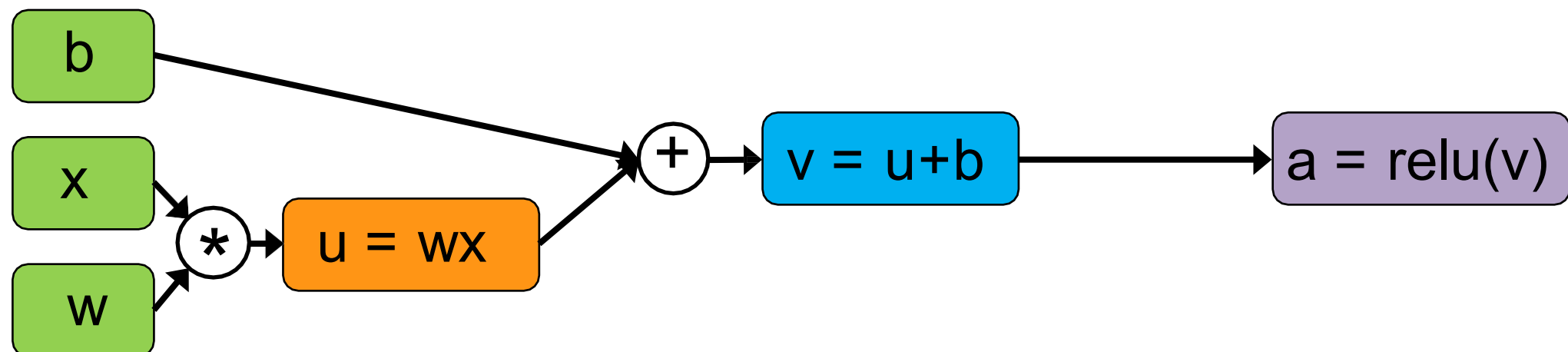
Suppose we have the following activation function:



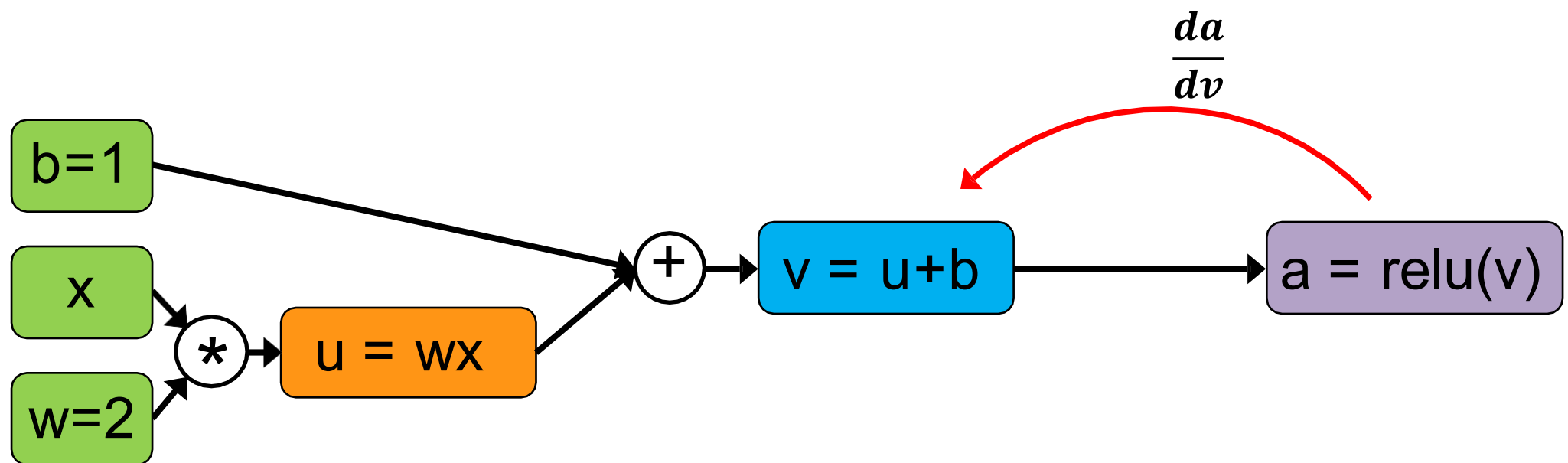
Computation Graphs

$$a(x, w, b) = \text{relu}(w \cdot x + b)$$

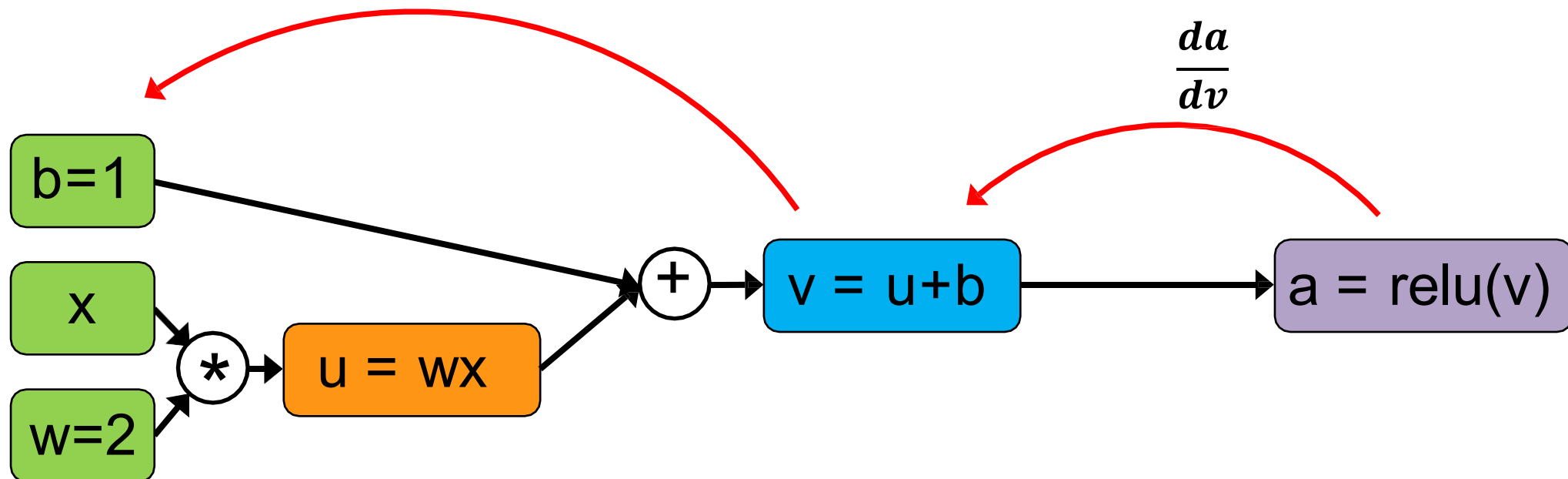
$$\underbrace{\underbrace{w \cdot x}_u + b}_v$$



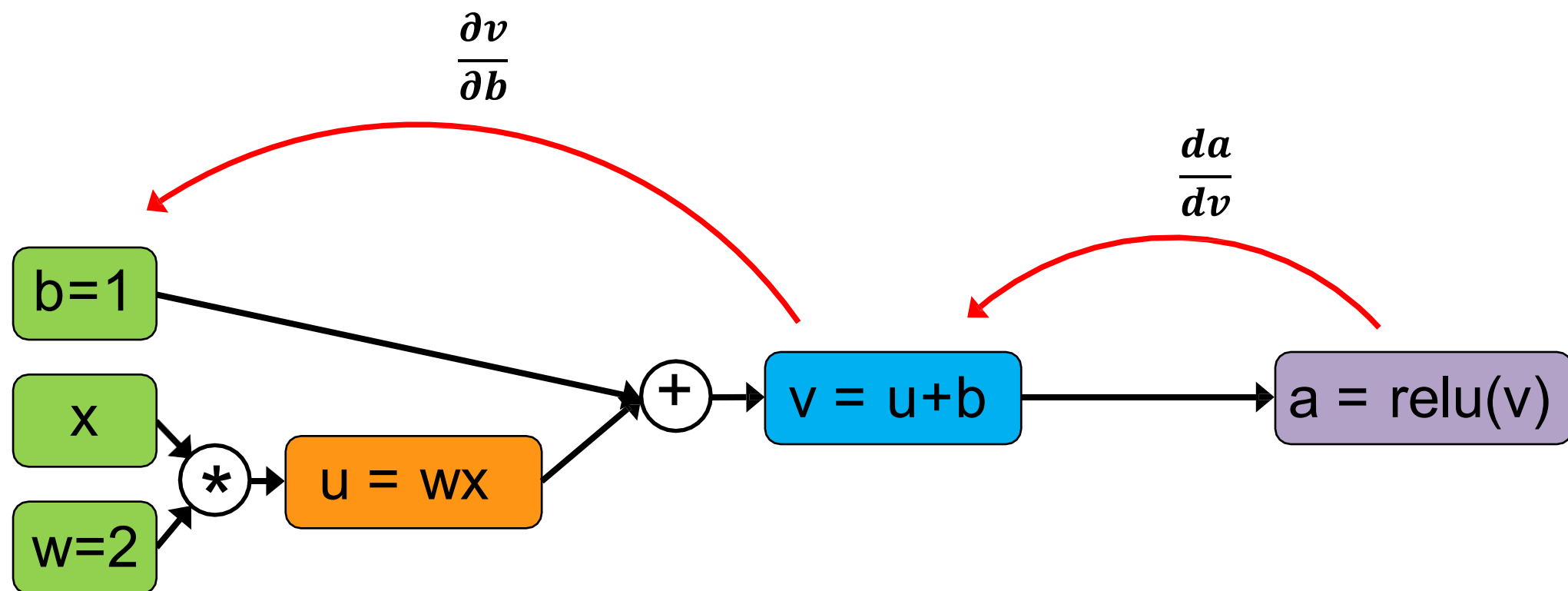
Computation Graphs



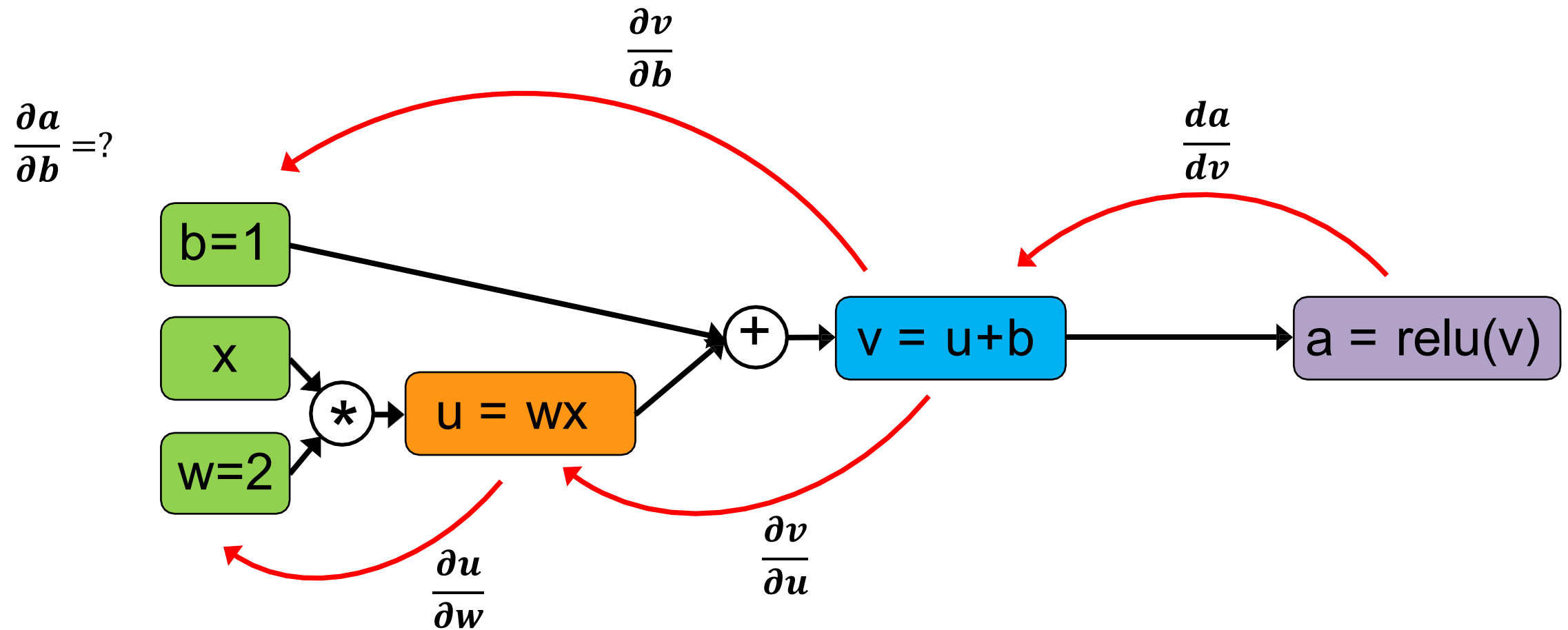
Computation Graphs



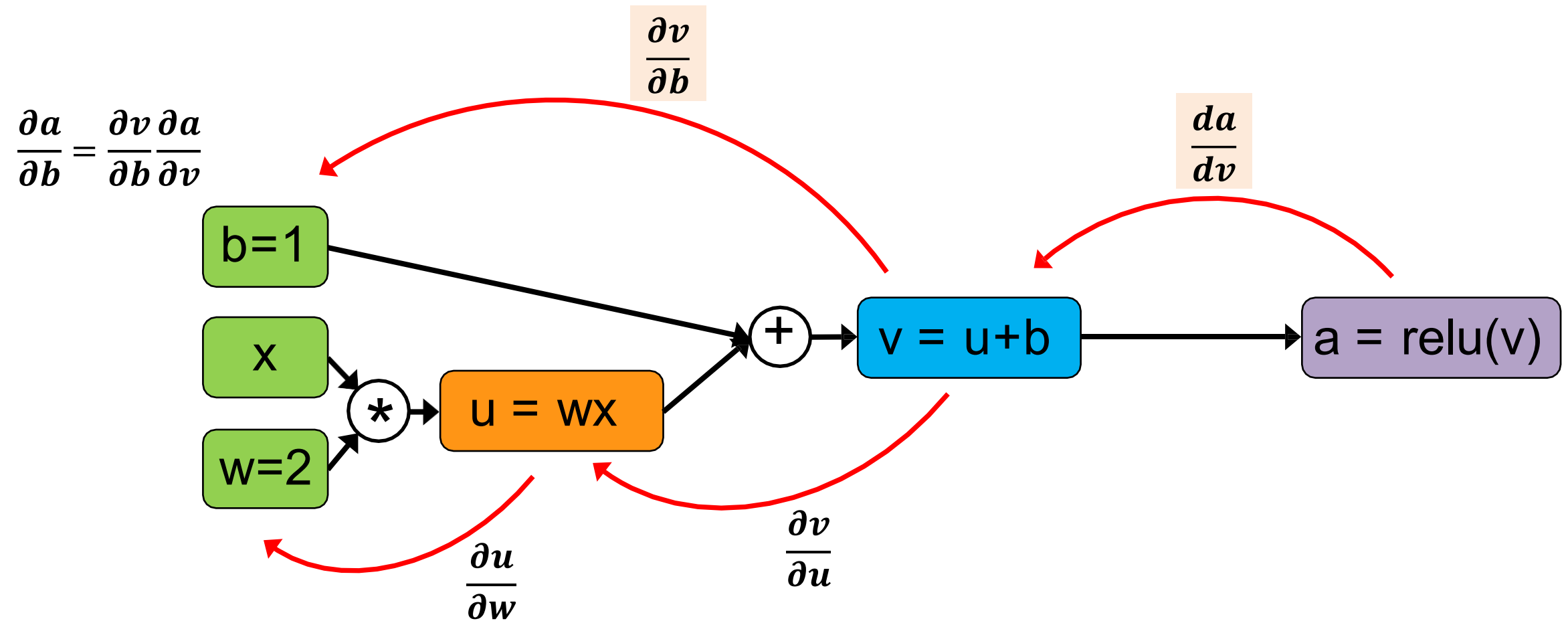
Computation Graphs



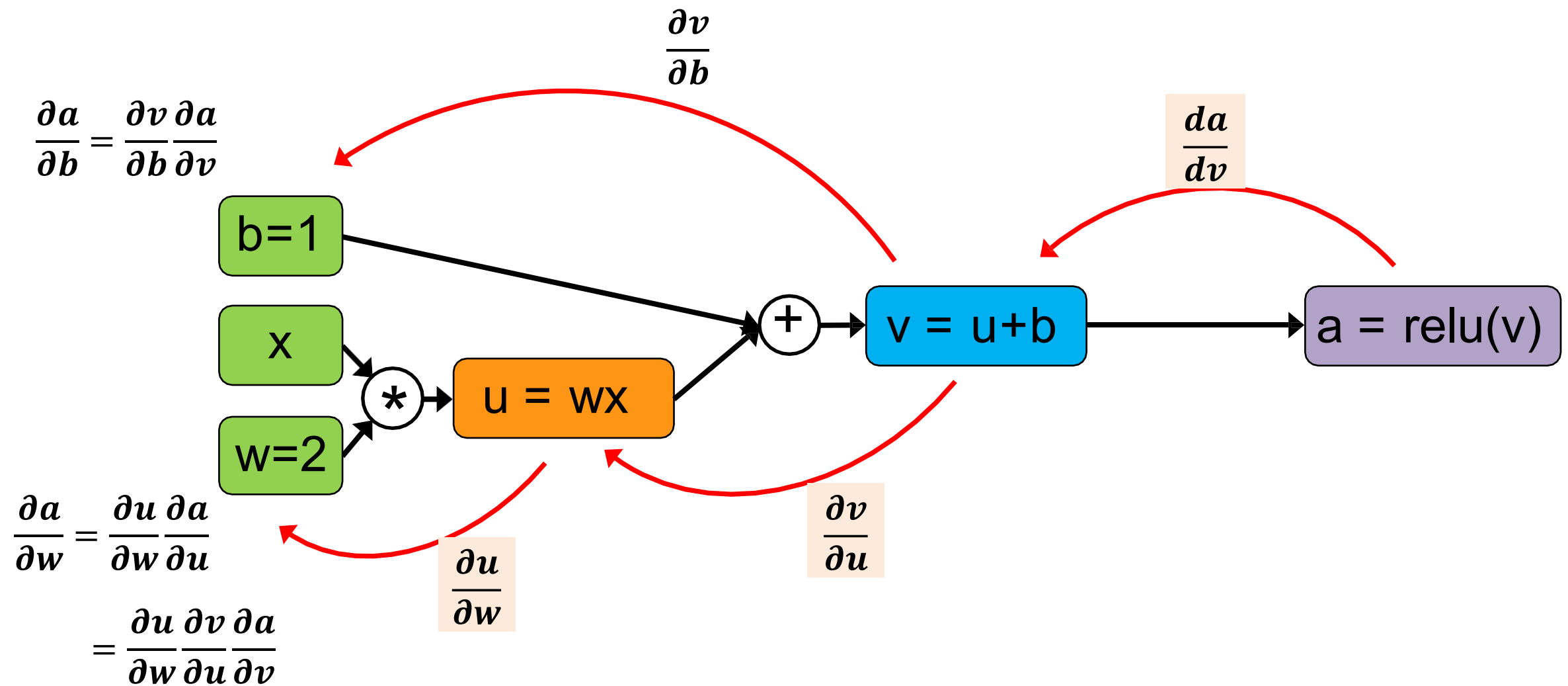
Computation Graphs



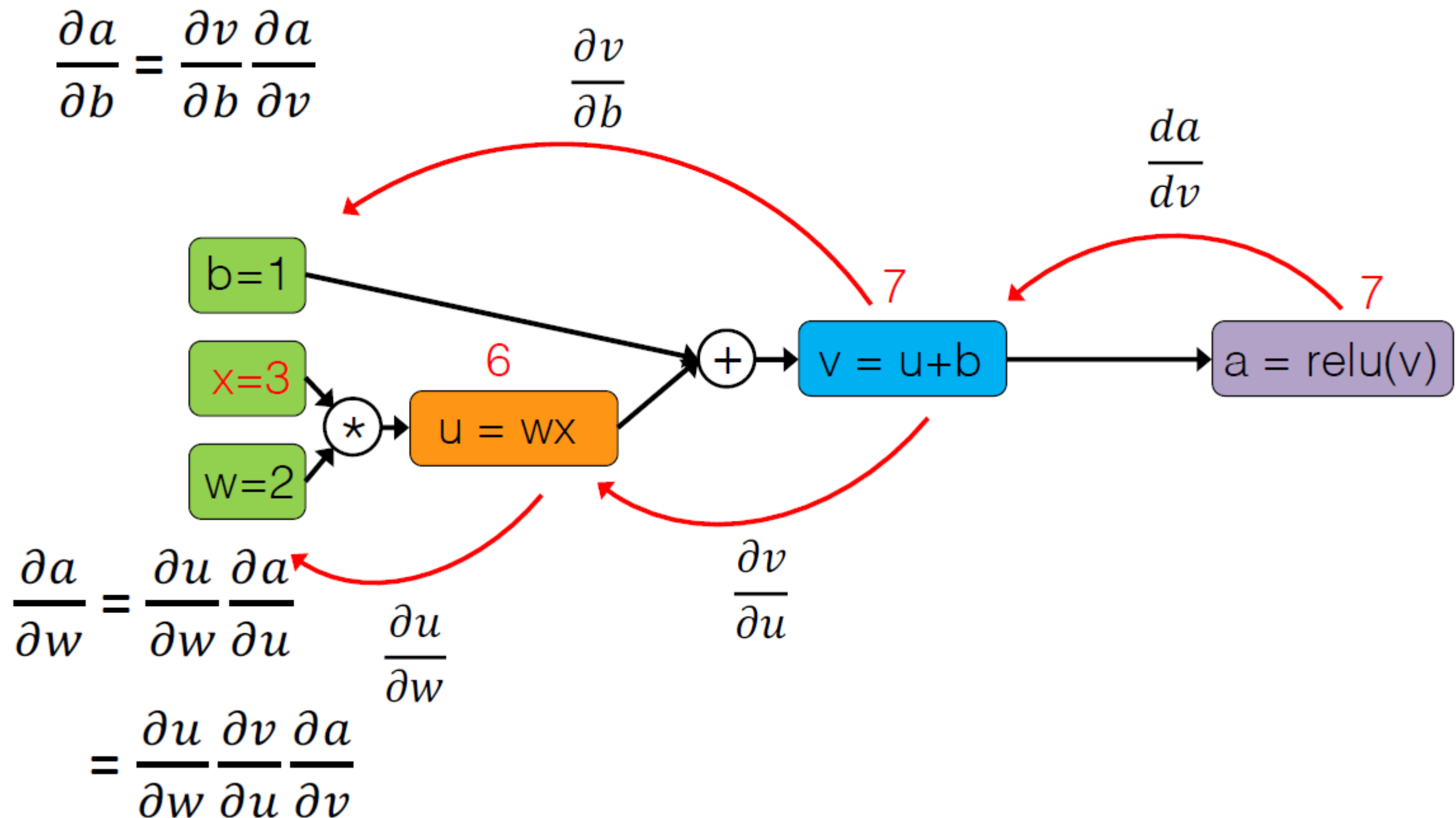
Computation Graphs



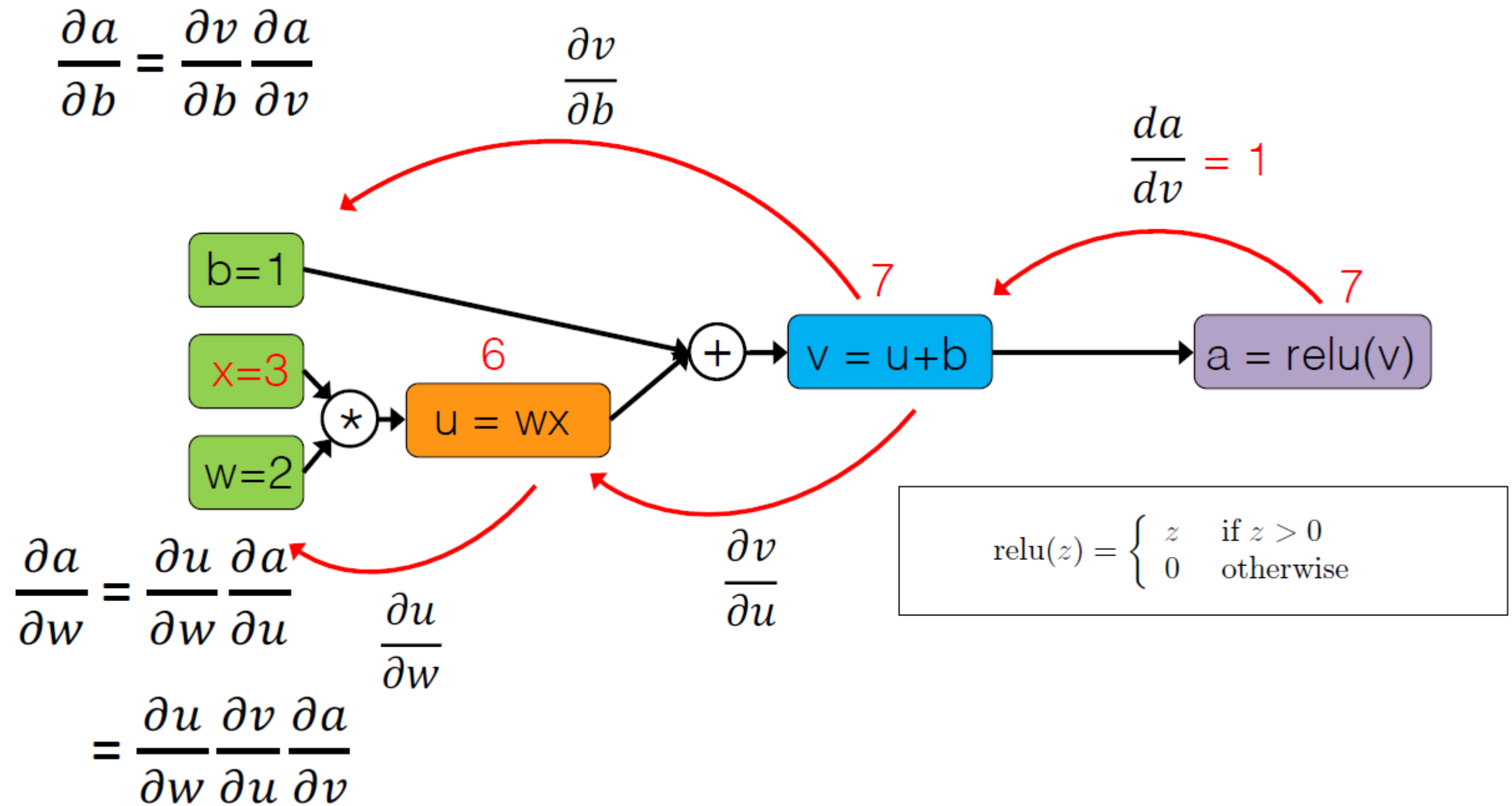
Computation Graphs



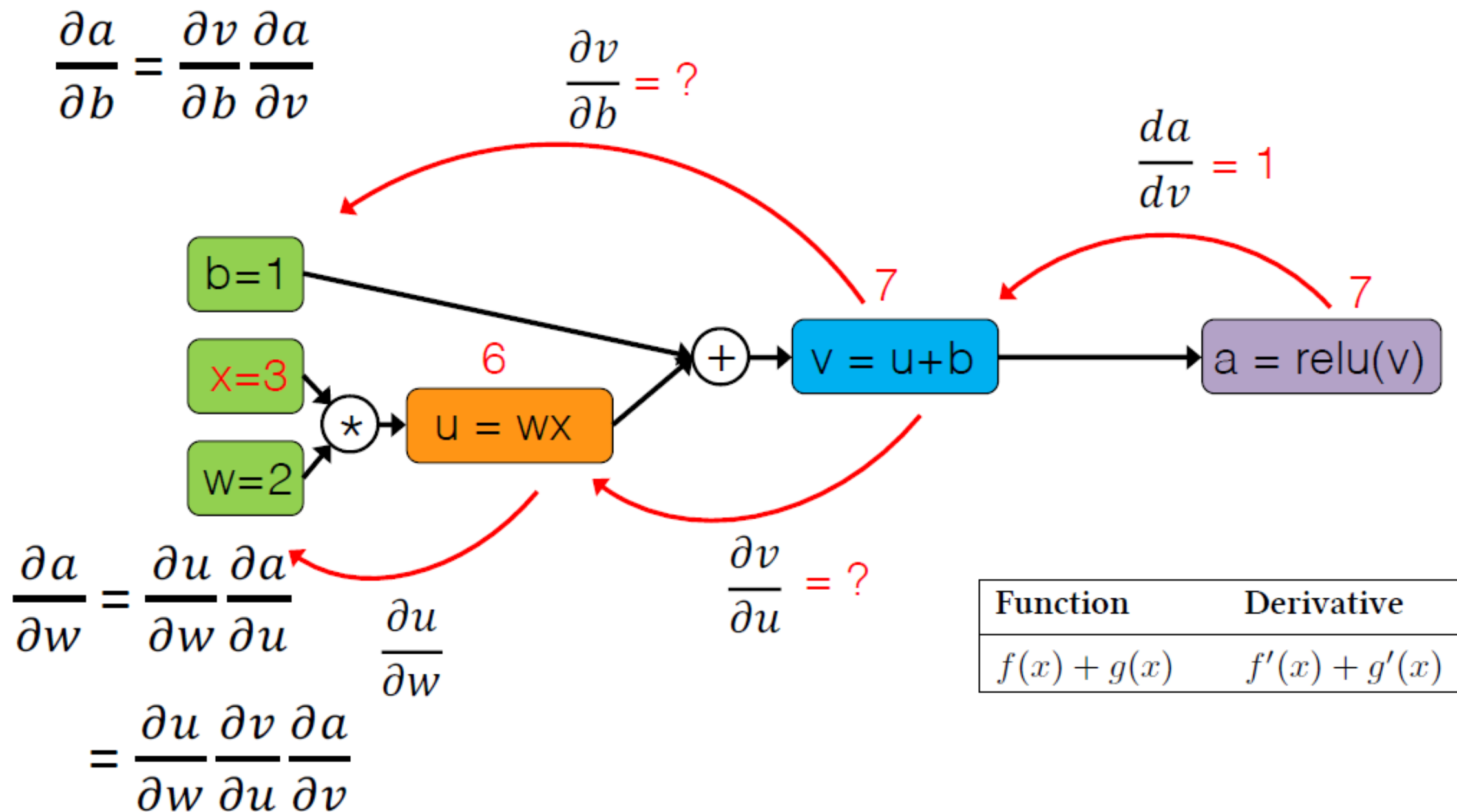
Computation Graphs



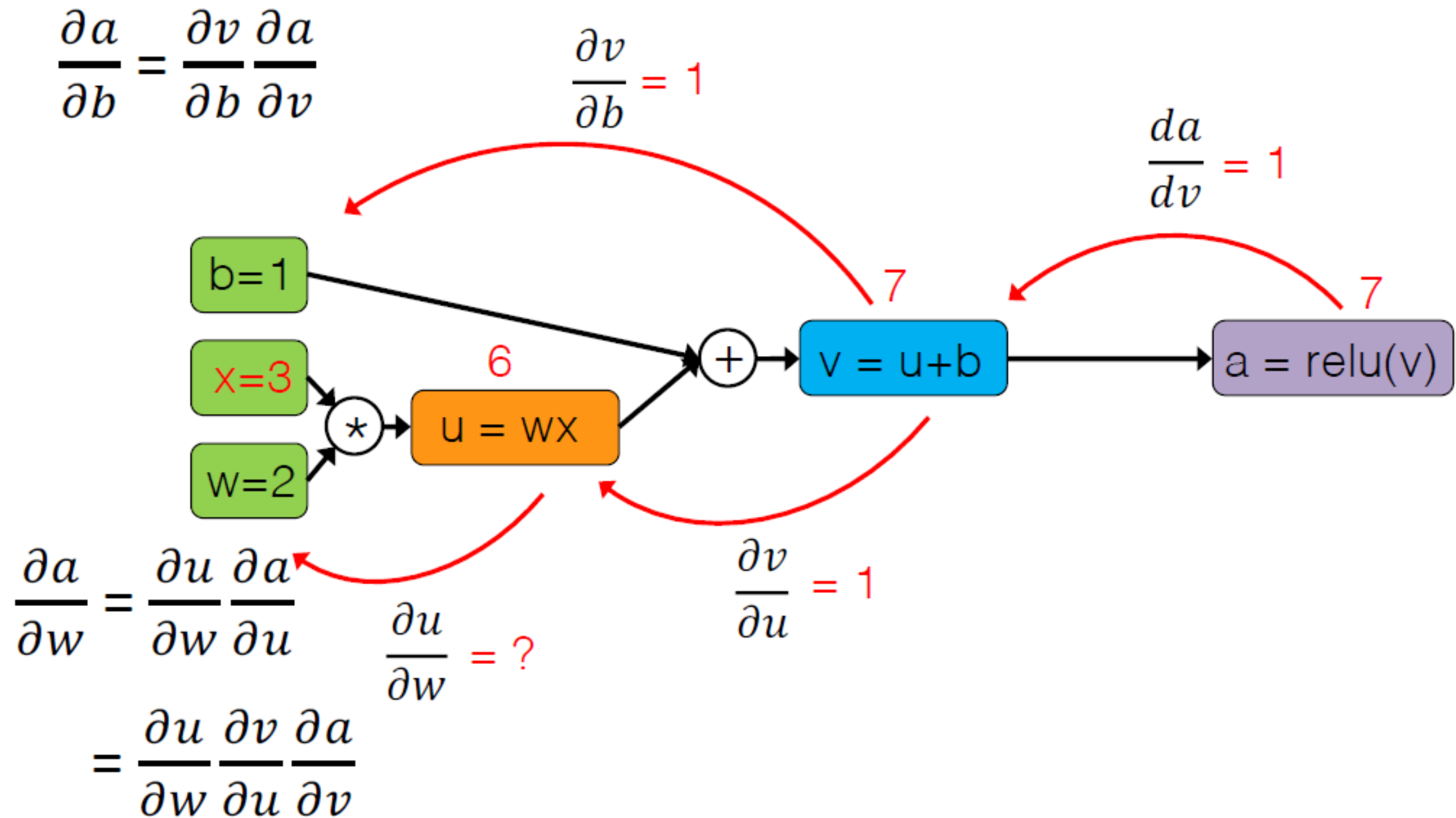
Computation Graphs



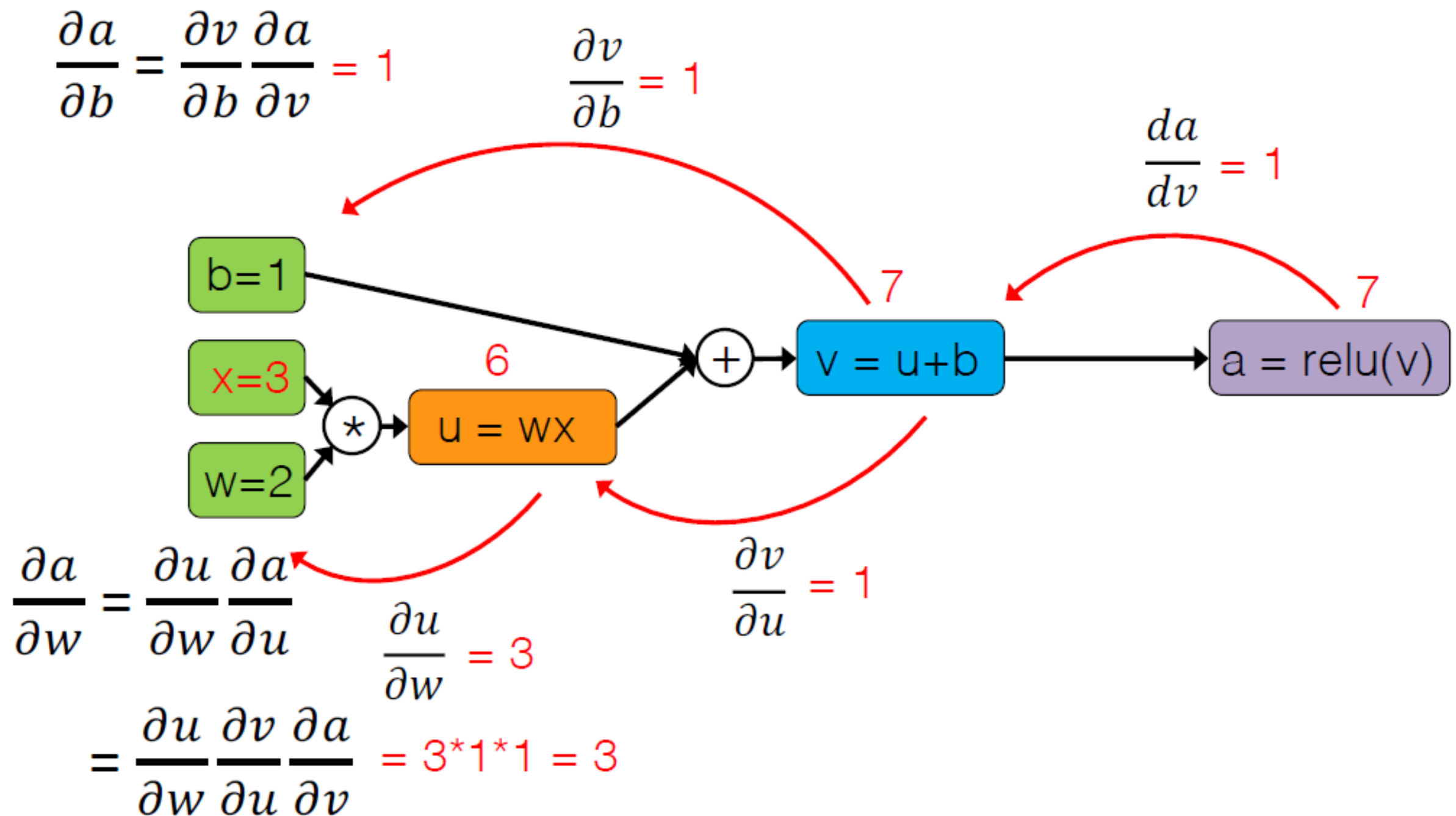
Computation Graphs



Computation Graphs



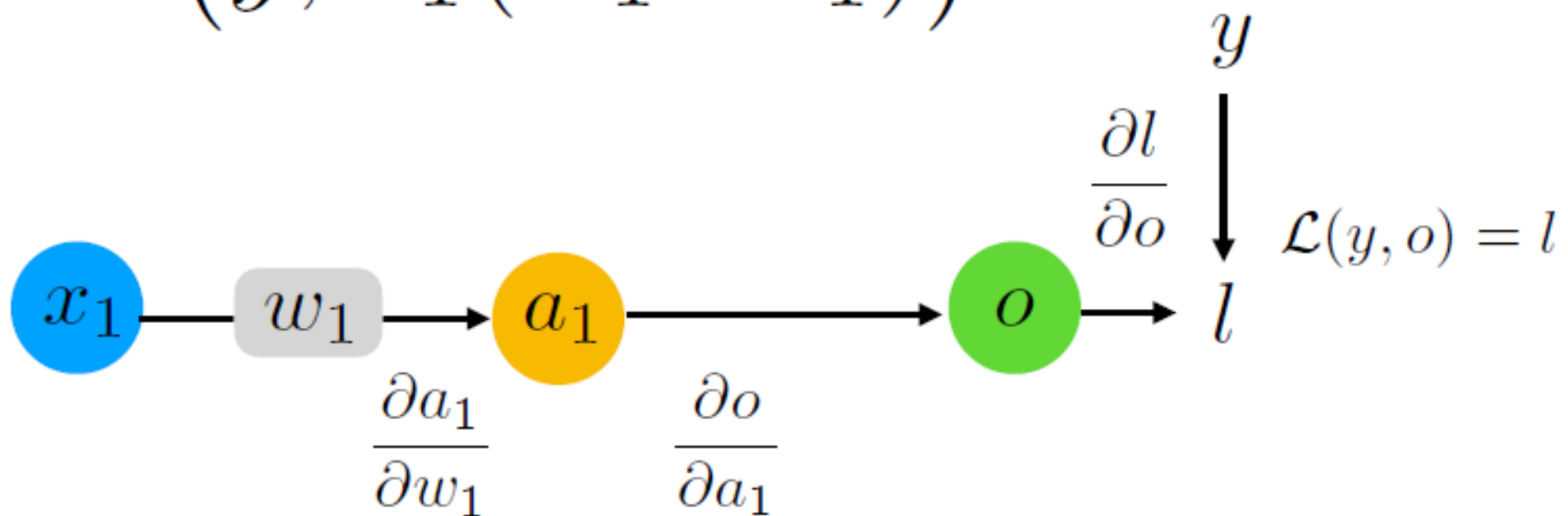
Computation Graphs



Some More Computation Graphs

Graph with Single Path

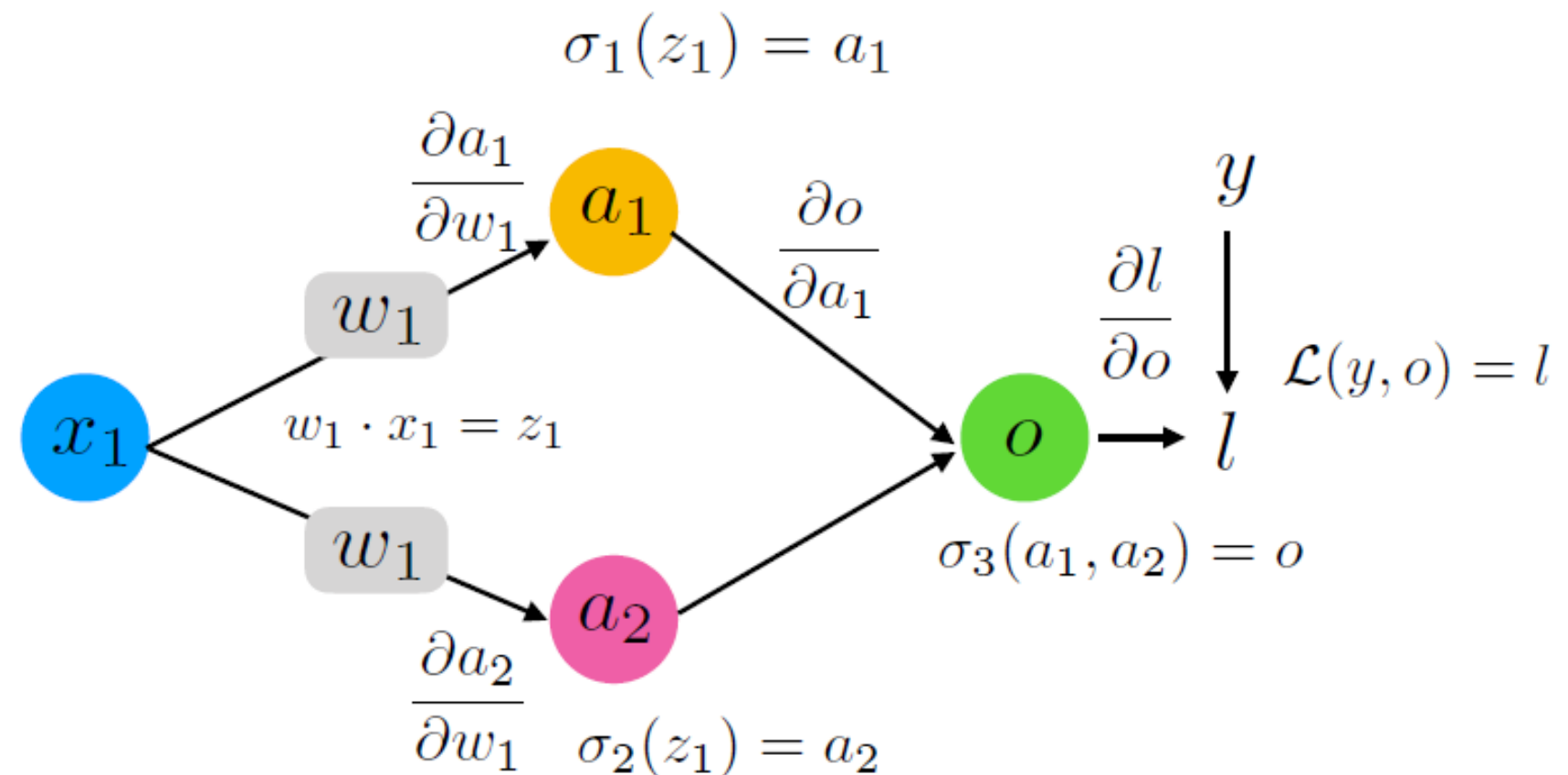
$$\mathcal{L}(y, \sigma_1(w_1 \cdot x_1))$$



$$\frac{\partial l}{\partial w_1} = \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_1} \quad (\text{univariate chain rule})$$

Graph with Weight Sharing

$$\mathcal{L}(y, \sigma_3[\sigma_1(w_1 \cdot x_1), \sigma_2(w_1 \cdot x_1)])$$

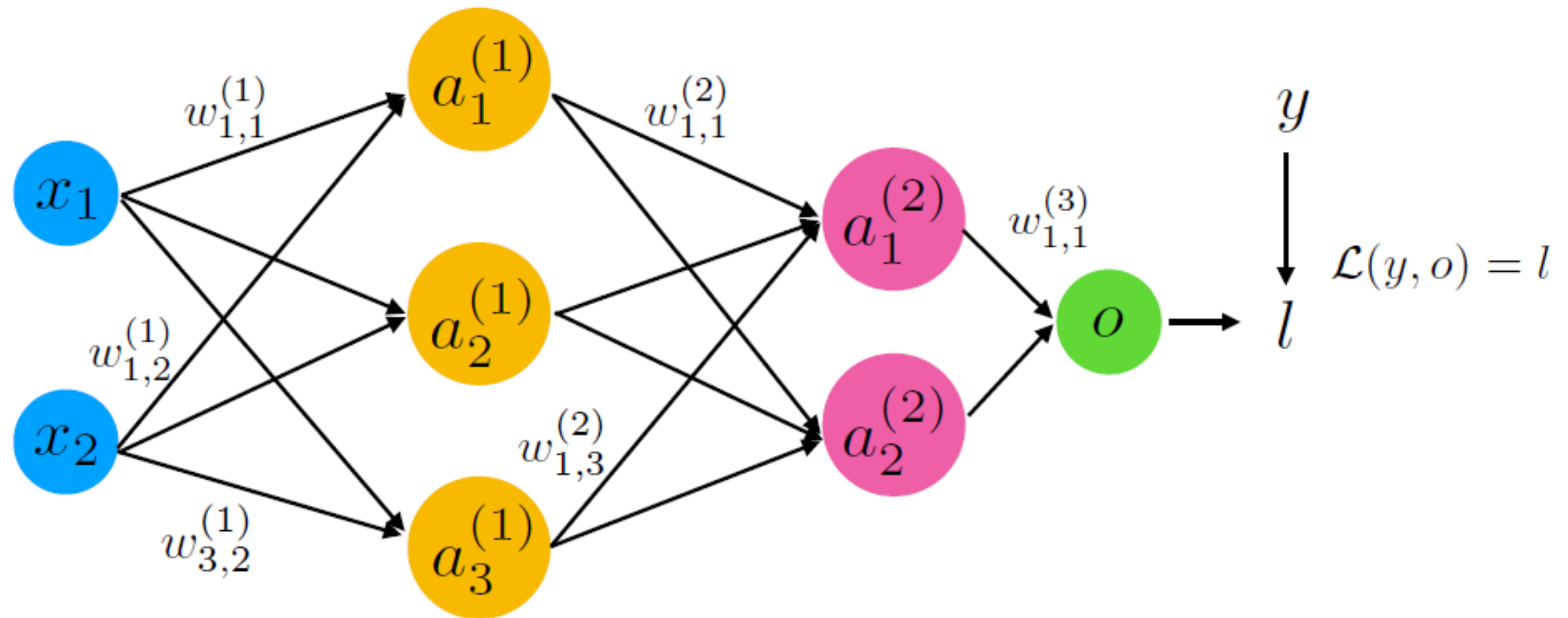


Upper path

$$\frac{\partial l}{\partial w_1} = \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_1} + \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_2} \cdot \frac{\partial a_2}{\partial w_1} \quad (\text{multivariable chain rule})$$

Lower path

Graph with Fully-Connected Layers (later in this course)



$$\begin{aligned} \frac{\partial l}{\partial w_{1,1}^{(1)}} &= \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{1,1}^{(1)}} \\ &\quad + \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_2^{(2)}} \cdot \frac{\partial a_2^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{1,1}^{(1)}} \end{aligned}$$

Automatic Differentiation with PyTorch

-- An Autograd Example

1. Online, batch, and minibatch mode
2. Relation between perceptron and linear regression
3. An iterative training algorithm for linear regression
4. Understanding gradient descent
5. Training an adaptive linear neuron (Adaline)
6. PyTorch Resources
7. Computation Graphs
- 8. Automatic Differentiation in PyTorch**
9. Training ADALINE Manually Vs Automatically in PyTorch
10. A Closer Look at the PyTorch API

Training an Adaptive Linear Neuron in PyTorch

1. Online, batch, and minibatch mode
2. Relation between perceptron and linear regression
3. An iterative training algorithm for linear regression
4. Understanding gradient descent
5. Training an adaptive linear neuron (Adaline)
6. PyTorch Resources
7. Computation Graphs
8. Automatic Differentiation in PyTorch
- 9. Training ADALINE Manually Vs Automatically in PyTorch**
10. A Closer Look at the PyTorch API

Using PyTorch: A Closer Look at the Object-Oriented and Functional APIs

1. Online, batch, and minibatch mode
2. Relation between perceptron and linear regression
3. An iterative training algorithm for linear regression
4. Understanding gradient descent
5. Training an adaptive linear neuron (Adaline)
6. PyTorch Resources
7. Computation Graphs
8. Automatic Differentiation in PyTorch
9. Training ADALINE Manually Vs Automatically in PyTorch
- 10. A Closer Look at the PyTorch API**

PyTorch Usage: Step 1 (Definition)

```
class MultilayerPerceptron(torch.nn.Module):  
  
    def __init__(self, num_features, num_classes):  
        super(MultilayerPerceptron, self).__init__()  
  
        ### 1st hidden layer  
        self.linear_1 = torch.nn.Linear(num_feat, num_h1)  
  
        ### 2nd hidden layer  
        self.linear_2 = torch.nn.Linear(num_h1, num_h2)  
  
        ### Output layer  
        self.linear_out = torch.nn.Linear(num_h2, num_classes)  
  
    def forward(self, x):  
        out = self.linear_1(x)  
        out = F.relu(out)  
        out = self.linear_2(out)  
        out = F.relu(out)  
        logits = self.linear_out(out)  
        probas = F.log_softmax(logits, dim=1)  
        return logits, probas
```

Backward will be inferred automatically if we use the nn.Module class!

Define model parameters that will be instantiated when created an object of this class

Define how and in what order the model parameters should be used in the forward pass

PyTorch Usage: Step 2 (Creation)

```
torch.manual_seed(random_seed)
```

```
model = MultilayerPerceptron(num_features=num_features,  
                             num_classes=num_classes)
```

Instantiate model
(creates the model parameters)

```
model = model.to(device)
```

```
optimizer = torch.optim.SGD(model.parameters(),  
                             lr=learning_rate)
```

Define an optimization method

PyTorch Usage: Step 2 (Creation)

```
torch.manual_seed(random_seed)
model = MultilayerPerceptron(num_features=num_features,
                             num_classes=num_classes)
```

```
model = model.to(device)
```

```
optimizer = torch.optim.SGD(model.parameters(),
                             lr=learning_rate)
```

← Optionally move model to GPU, where
device e.g. `torch.device('cuda:0')`

PyTorch Usage: Step 3 (Training)

```
for epoch in range(num_epochs):
    model.train()
    for batch_idx, (features, targets) in enumerate(train_loader):

        features = features.view(-1, 28*28).to(device)
        targets = targets.to(device)

        ### FORWARD AND BACK PROP
        logits, probas = model(features)
        cost = F.cross_entropy(probas, targets)
        optimizer.zero_grad()

        cost.backward()

        ### UPDATE MODEL PARAMETERS
        optimizer.step()

    model.eval()
    with torch.no_grad():
        # compute accuracy
```

Run for a specified number of epochs

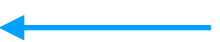
Iterate over minibatches in epoch

If your model is on the GPU, data should also be on the GPU

`y = model(x)` calls `__call__` and then `.forward()`, where some extra stuff is done in `_call__`;
don't run `y = model.forward(x)` directly

Gradients at each leaf node are accumulated under the `.grad` attribute, not just stored. This is why we have to zero them before each backward pass

PyTorch Usage: Step 3 (Training)

```
for epoch in range(num_epochs):  
    model.train()  
    for batch_idx, (features, targets) in enumerate(train_loader):  
  
        features = features.view(-1, 28*28).to(device)  
        targets = targets.to(device)  
  
        ### FORWARD AND BACK PROP  
        logits, probas = model(features)  This will run the forward() method  
        loss = F.cross_entropy(logits, targets)  Define a loss function to optimize  
        optimizer.zero_grad()  Set the gradient to zero  
                                (could be non-zero from a previous forward pass)  
        loss.backward()   
  
        ### UPDATE MODEL PARAMETERS  
        optimizer.step()  Compute the gradients, the backward is  
                                automatically constructed by "autograd" based on  
                                the forward() method and the loss function  
  
    model.eval()  
    with torch.no_grad():  
        # compute accuracy  Use the gradients to update the weights according to  
                                the optimization method (defined on the previous  
                                slide)  
                                E.g., for SGD,  $w := w + \text{learning\_rate} \times \text{gradient}$ 
```

PyTorch Usage: Step 3 (Training)

```
for epoch in range(num_epochs):
    model.train()
    for batch_idx, (features, targets) in enumerate(train_loader):

        features = features.view(-1, 28*28).to(device)
        targets = targets.to(device)

        ### FORWARD AND BACK PROP
        logits, probas = model(features)
        loss = F.cross_entropy(logits, targets)
        optimizer.zero_grad()

        loss.backward()

        ### UPDATE MODEL PARAMETERS
        optimizer.step()

    model.eval()
    with torch.no_grad():
        # compute accuracy
```

For evaluation, set the model to eval mode (will be relevant later when we use Dropout or BatchNorm)

This prevents the computation graph for backpropagation from automatically being build in the background to save memory

PyTorch Usage: Step 3 (Training)

```
for epoch in range(num_epochs):  
    model.train()  
    for batch_idx, (features, targets) in enumerate(train_loader):  
  
        features = features.view(-1, 28*28).to(device)  
        targets = targets.to(device)  
  
        ### FORWARD AND BACK PROP  
        logits, probas = model(features)  
        loss = F.cross_entropy(logits, targets)  
        optimizer.zero_grad()  
  
        loss.backward()  
  
        ### UPDATE MODEL PARAMETERS  
        optimizer.step()  
  
    model.eval()  
    with torch.no_grad():  
        # compute accuracy
```



logits because of computational efficiency.
Basically, it internally uses a `log_softmax(logits)` function that is more stable than `log(softmax(logits))`.
More on logits ("net inputs" of the last layer) in the next lecture. Please also see

Objected-Oriented vs Functional* API

*Note that with "functional" I mean "functional programming" (one paradigm in CS)

`torch.nn.functional` = api without internal state

```
import torch.nn.functional as F
```

```
class MultilayerPerceptron(torch.nn.Module):  
  
    def __init__(self, num_features, num_classes):  
        super(MultilayerPerceptron, self).__init__()  
  
        ### 1st hidden layer  
        self.linear_1 = torch.nn.Linear(num_features,  
                                         num_hidden_1)  
  
        ### 2nd hidden layer  
        self.linear_2 = torch.nn.Linear(num_hidden_1,  
                                         num_hidden_2)  
  
        ### Output layer  
        self.linear_out = torch.nn.Linear(num_hidden_2,  
                                           num_classes)  
  
    def forward(self, x):  
        out = self.linear_1(x)  
        out = F.relu(out)  
        out = self.linear_2(out)  
        out = F.relu(out)  
        logits = self.linear_out(out)  
        probas = F.log_softmax(logits, dim=1)  
        return logits, probas
```

Unnecessary because these functions
don't need to store a state but maybe
helpful for keeping track of order of ops
(when implementing "forward")

```
class MultilayerPerceptron(torch.nn.Module):
```

```
    def __init__(self, num_features, num_classes):  
        super(MultilayerPerceptron, self).__init__()  
  
        ### 1st hidden layer  
        self.linear_1 = torch.nn.Linear(num_features,  
                                         num_hidden_1)  
  
        self.relu1 = torch.nn.ReLU()  
  
        ### 2nd hidden layer  
        self.linear_2 = torch.nn.Linear(num_hidden_1,  
                                         num_hidden_2)  
  
        self.relu2 = torch.nn.ReLU()  
  
        ### Output layer  
        self.linear_out = torch.nn.Linear(num_hidden_2,  
                                           num_classes)  
  
        self.softmax = torch.nn.Softmax()  
  
    def forward(self, x):  
        out = self.linear_1(x)  
        out = self.relu1(out)  
        out = self.linear_2(out)  
        out = self.relu2(out)  
        logits = self.linear_out(out)  
        probas = self.softmax(logits, dim=1)  
        return logits, probas
```

Objected-Oriented vs Functional API

Using "Sequential"

```
import torch.nn.functional as F
```

```
class MultilayerPerceptron(torch.nn.Module):  
  
    def __init__(self, num_features, num_classes):  
        super(MultilayerPerceptron, self).__init__()  
  
        ### 1st hidden layer  
        self.linear_1 = torch.nn.Linear(num_features,  
                                         num_hidden_1)  
  
        ### 2nd hidden layer  
        self.linear_2 = torch.nn.Linear(num_hidden_1,  
                                         num_hidden_2)  
  
        ### Output layer  
        self.linear_out = torch.nn.Linear(num_hidden_2,  
                                           num_classes)  
  
    def forward(self, x):  
        out = self.linear_1(x)  
        out = F.relu(out)  
        out = self.linear_2(out)  
        out = F.relu(out)  
        logits = self.linear_out(out)  
        probas = F.log_softmax(logits, dim=1)  
        return logits, probas
```

```
class MultilayerPerceptron(torch.nn.Module):  
  
    def __init__(self, num_features, num_classes):  
        super(MultilayerPerceptron, self).__init__()  
  
        self.my_network = torch.nn.Sequential(  
            torch.nn.Linear(num_features, num_hidden_1),  
            torch.nn.ReLU(),  
            torch.nn.Linear(num_hidden_1, num_hidden_2),  
            torch.nn.ReLU(),  
            torch.nn.Linear(num_hidden_2, num_classes)  
        )  
  
    def forward(self, x):  
        logits = self.my_network(x)  
        probas = F.softmax(logits, dim=1)  
        return logits, probas
```

Much more compact and clear, but
"forward" may be harder to debug if there
are errors (we cannot simply add
breakpoints or insert "print" statements)

Objected-Oriented vs Functional API

Using "Sequential"

1)

```
class MultilayerPerceptron(torch.nn.Module):  
  
    def __init__(self, num_features, num_classes):  
        super(MultilayerPerceptron, self).__init__()  
  
        self.my_network = torch.nn.Sequential(  
            torch.nn.Linear(num_features, num_hidden),  
            torch.nn.ReLU(),  
            torch.nn.Linear(num_hidden_1, num_hidden_2),  
            torch.nn.ReLU(),  
            torch.nn.Linear(num_hidden_2, num_classes)  
        )  
  
    def forward(self, x):  
        logits = self.my_network(x)  
        probas = F.softmax(logits, dim=1)  
        return logits, probas
```

Much more compact and clear, but
"forward" may be harder to debug if there
are errors (we cannot simply add
breakpoints or insert "print" statements)

2)

However, if you use Sequential, you can
define "hooks" to get intermediate outputs.
For example:

```
[7]: model.net  
[7]: Sequential(  
  (0): Linear(in_features=784, out_features=128, bias=True)  
  (1): ReLU(inplace)  
  (2): Linear(in_features=128, out_features=256, bias=True)  
  (3): ReLU(inplace)  
  (4): Linear(in_features=256, out_features=10, bias=True)  
)  
[ ]: If we want to get the output from the 2nd layer during the forward pass, we can register a hook as follows:  
[8]: outputs = []  
      def hook(module, input, output):  
          outputs.append(output)  
      model.net[2].register_forward_hook(hook)  
[8]: <torch.utils.hooks.RemovableHandle at 0x7f659c6685c0>  
      Now, if we call the model on some inputs, it will save the intermediate results in the "outputs" list:  
[9]: _ = model(features)  
      print(outputs)  
      [tensor([[0.5341, 1.0513, 2.3542, ..., 0.0000, 0.0000, 0.0000],  
                [0.0000, 0.6676, 0.6620, ..., 0.0000, 0.0000, 2.4056],  
                [1.1520, 0.0000, 0.0000, ..., 2.5860, 0.8992, 0.9642],  
                ...,  
                [0.0000, 0.1076, 0.0000, ..., 1.8367, 0.0000, 2.5203],  
                [0.5415, 0.0000, 0.0000, ..., 2.7968, 0.8244, 1.6335],  
                [1.0710, 0.9805, 3.0103, ..., 0.0000, 0.0000, 0.0000]],  
              device='cuda:3', grad_fn=<ThresholdBackward1>)]
```

Jupyter Notebook vs Python Scripts

In general, we recommend to use jupyter notebooks for initial exploration/ playing around with new models and code. Python scripts should be used as soon as you want to train the model on a bigger dataset where also reproducibility is more important.

Our recommended workflow:

1. Start with a jupyter notebook
2. Explore the data and models
3. Build your classes/ methods inside cells of the notebook
4. Move your code to python scripts
5. Train/ deploy on server

Jupyter Notebook	Python Scripts
+ Exploration	+ Running longer jobs without interruption
+ Debugging	+ Easy to track changes with git
- Can become a huge file	- Debugging mostly means rerunning the whole script
- Can be interrupted (don't use for long training)	
- Prone to errors and become a mess	

Type	Convention	Example
Packages & Modules	lower_with_under	from prefetch_generator import BackgroundGenerator
Classes	CapWords	class DataLoader
Constants	CAPS_WITH_UNDER	BATCH_SIZE=16
Instances	lower_with_under	dataset = Dataset
Methods & Functions	lower_with_under()	def visualize_tensor()
Variables	lower_with_under	background_color ='Blue'

More PyTorch features will be introduced step-by-step later in this course when we start working with more complex networks, including

- Running code on the GPU
- Using efficient data loaders
- Splitting networks across different GPUs